

STUDENT ATTENDANCE ANALYSIS USING MACHINE LEARNING

A PROJECT REPORT

Submitted by

LAKSHAYA. M

820422205032

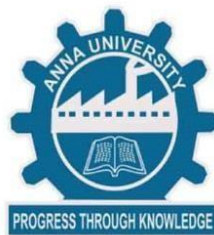
In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY



**ANJALAI AMMAL MAHALINGAM ENGINEERING
COLLEGE**

ANNA UNIVERSITY: CHENNAI 600025

MAY 2026

ANNA UNIVERSITY: CHENNAI 600025

BONAFIDE CERTIFICATE

Certified that this Project report “**STUDENT ATTENDANCE ANALYSIS USING MACHINE LEARNING**” is the bonafide work of “**LAKSHAYA.M - 820422205032**” who carried out the Project work under my supervision.

SIGNATURE

Dr. K. Mohan, M.Tech, Ph.D.,

HEAD OF THE DEPARTMENT

Department Of Information
Technology,

Anjalai Ammal Mahalingam
Engineering College,

Kovilvenni – 614 403,
Thiruvarur District.

SIGNATURE

Dr. R. Rebekha, M.Tech, Ph.D,

SUPERVISOR

Assistant Professor,
Department Of Information
Technology,

Anjalai Ammal Mahalingam
Engineering College,

Kovilvenni – 614 403, Thiruvarur
District.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

First and foremost, we intensively triumph in the work of the Almighty's hands with whose grace and power this endeavor has been a success.

We would like to express our sincere gratitude to the **Managing Trustee Justice A. Ramamurthy** and the Management of **Anjalai Ammal Mahalingam Engineering College** for providing adequate infrastructure facilities to learn the latest technologies and complete our project.

We take great pleasure in expressing our sincere thanks to our principal **Dr. S.N. Ramaswamy, Ph.D.**, for his valuable support and encouragement regarding our project.

We feel happy to express our sincere thanks to **Dr. K. Mohan, M.Tech., Ph.D.**, Head of the Department (HOD) & Project Coordinator of Information Technology, for his encouragement and for having granted permission to work this project.

We would like to express our deep sense of gratitude to our guide **Dr.R.Rebekha, M.Tech., Ph.D, Assistant Professor**, Department of Information Technology, for spending his valuable time with us guiding throughout the project work. We also express our sincere thanks to all the IT department staff members and lab technicians.

We would like to thank **e-soft IT Solutions** and joint supervisor from that company, **Ms.V.P.Suchitra** for their valuable guidance, mentorship, and technical support throughout the project.

At the outset, we wish to express our sincere gratitude to our beloved parents and friends who are solely responsible for the successful completion of our B.Tech degree course.

ABSTRACT

Student attendance is a critical parameter influencing academic performance, institutional efficiency, and overall student well-being. Traditional manual attendance systems are not only time-consuming but are also highly susceptible to errors, proxy marking, and data inconsistencies. With the rapid advancement of Artificial Intelligence and Machine Learning technologies, there exists a significant opportunity to automate and enhance the accuracy of attendance tracking systems in educational institutions. This project presents the design and implementation of an intelligent Student Attendance Analysis System powered by Machine Learning algorithms. The system collects, processes, and analyses attendance data to identify patterns, predict at-risk students, and generate actionable insights for faculty and administrators. The core machine learning models employed include Random Forest, which form an ensemble approach and prediction. The proposed system integrates a face recognition-based automated attendance capture module with a backend database, enabling real-time data logging without manual intervention. The captured data is preprocessed using normalization and feature engineering techniques, and then fed into trained ML models to produce attendance trend analysis, student performance correlation, and early warning flags for low-attendance students. The system includes a comprehensive dashboard for teachers, department heads, and students, displaying visualizations such as attendance heatmaps, trend graphs, and department-wise comparative reports. Evaluation results demonstrate that the proposed system achieves an accuracy of over 94% in predicting attendance behaviour patterns, outperforming conventional threshold-based approaches.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
	LIST OF FIGURES	4
	LIST OF ABBREVIATIONS	5
1	INTRODUCTION	6
	1.1 Overview	7
	1.1.1 Need for the Project	8
	1.1.2 Objectives	9
	1.1.3 Scope of the Project	10
	1.2 Tools and Technologies Used	11
	1.2.1 Frontend: Python Flask	11
	1.2.2 Backend: MySQL Database	11
	1.2.3 Machine Learning Libraries	12
	1.3 ML Techniques Used	13
	1.3.1 Random Forest Algorithm	13
	1.3.2 Algorithm Configuration	13
	1.3.3 Random Forest for Attendance Prediction	14
	1.4 Literature Survey	16
2	PROBLEM ANALYSIS	21
	2.1 Problem Definition	22
	2.2 Existing System	22
	2.2.1 Disadvantages of Existing System	23
	2.3 Proposed System	24
	2.3.1 Advantages of the Proposed System	24
	2.4 System Modules	25

2.4.1 Configuration Module	25
2.4.2 Database Helper Module	26
2.4.3 ML Engine Module	27
2.4.4 Alert Engine Module	28
2.4.5 Web Dashboard Module	29
2.4.6 CLI Entry Point Module	29
2.4.7 Database Schema Module	30
2.5 Requirement Specifications	31
2.5.1 Minimum Requirements	31
2.5.2 Recommended Requirements	31
2.6 System Design	31
2.7 System Architecture	33
2.8 Data Flow Diagrams	36
2.8.1 DFD Level 0-Context Diagram	36
2.8.2 DFD Level 1	37
2.9 UML Diagrams	38
2.9.1 Use Case Diagram	38
2.9.2 Sequence Diagram	39
2.9.3 Class Diagram	40
2.9.4 Activity Diagram	41
2.10 Implementation	42
2.10.1 Configuration Module Implementation	42
2.10.2 Database Helper Module	43
2.10.3 Attendance Processing Module	44
2.10.4 Machine Learning Model Training	45
Implementation	
2.10.5 Machine Learning Prediction	46
Implementation	

	2.10.6 Alert System Implementation	47
	2.10.7 Web Application Implementation	47
	2.10.8 CLI Implementation	48
	2.10.9 Database Implementation	48
	2.10.10 Algorithms	48
	2.10.11 Pseudocode	52
	2.10.12 Module Flow Explanation and Source Code	54
	2.11 Testing	78
	2.11.1 Unit Testing	85
	2.11.2 Integration Testing	86
	2.11.3 Performance Testing	87
	2.11.4 Security Testing	88
	2.11.5 Usability Testing	89
	2.12 Results and Output	91
	2.13 Screenshots Explanation	93
	2.13.1 Dashboard Home Page	93
	2.13.2 ML Prediction Result	94
	2.13.3 Alert Notification Screen	95
3	CONCLUSION AND FUTURE ENHANCEMENTS	88
	3.1 Conclusion	97
	3.2 Future Enhancements	98
	REFERENCES	99
	APPENDIX	100

ANNEXURE 1	109
Company Details	110
ANNEXURE 2	113
Conference Details	114

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
2.7	System Architecture Diagram	29
2.8.1	DFD Level 0 Diagram	30
2.8.2	DFD Level 1 Diagram	31
2.9.1	Use Case Diagram	32
2.9.2	Sequence Diagram	33
2.9.3	Class Diagram	34
2.9.4	Activity Diagram	35
2.11.1	Db and Seeding Testing	78
2.11.2	Modal and prediction Testing	79
2.11.3	Dry run and Threshold Testing	80
2.11.4	API and sms simulation Testing	82
2.11.5	All Testing Results	82
2.11.6	Testing	83
2.13.1	Dashboard Home Page	85
2.13.2	ML Prediction Results	86
2.13.3	Alert Notification Screen	87

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
ML	Machine Learning
AI	Artificial Intelligence
SVM	Support Vector Machine
KNN	K-Nearest Neighbors
CNN	Convolutional Neural Network
RF	Random Forest
DT	Decision Tree
LR	Logistic Regression
CV	Computer Vision
OCR	Optical Character Recognition
API	Application Programming Interface
UI	User Interface
UML	Unified Modeling Language
DFD	Data Flow Diagram
SQL	Structured Query Language
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
HTTP	HyperText Transfer Protocol
HOD	Head of the Department
RFID	Radio Frequency Identification
IoT	Internet of Things
GUI	Graphical User Interface

CHAPTER 1
INTRODUCTION

1.1 Overview

Student attendance is widely recognized as one of the most significant predictors of academic success in educational institutions. Research consistently shows that students with regular attendance demonstrate better academic performance, higher retention rates, and improved overall learning outcomes. However, managing and analysing attendance data effectively remains a persistent challenge for institutions operating at scale.

Conventional attendance recording methods, which typically involve manual roll calls or paper-based registers, are not only labour-intensive but are also prone to human error, proxy fraud, and data loss. As educational institutions increasingly adopt digital infrastructure, there is a growing need for intelligent, automated systems that can not only record attendance but also derive meaningful insights from that data.

The advent of Machine Learning (ML) has opened new frontiers in educational data analytics. By applying supervised and unsupervised learning algorithms to historical attendance data, it becomes possible to identify behavioural patterns, detect anomalies, predict future attendance trends, and flag at-risk students early enough for timely intervention. This predictive capability is particularly valuable for academic counsellors and faculty members who need actionable intelligence to guide student support initiatives.

This project presents a comprehensive Student Attendance Analysis System that leverages machine learning to transform raw attendance data into rich, decision-supporting insights. The system integrates face recognition-based automated attendance capture with a robust ML-powered analytics engine to deliver a seamless, accurate, and intelligent attendance management solution.

1.1.1 Need for the Project

The increasing complexity of modern educational environments calls for smarter approaches to student management. The need for this project arises from several critical challenges observed in contemporary academic settings:

- Manual attendance processes consume significant faculty time that could be better allocated to teaching and mentoring activities.
- Paper-based records are susceptible to damage, loss, and tampering, compromising the integrity of attendance data.
- Traditional systems do not provide any analytical capability, making it difficult to identify trends or patterns in student behaviour.
- The absence of early warning systems means that students who are developing attendance problems are not identified until the situation becomes severe.
- Parents and guardians typically have no real-time visibility into their ward's attendance, limiting their ability to intervene proactively.
- Generating attendance reports for examinations, scholarships, or compliance purposes is time-consuming and error-prone when done manually.
- With the rise of hybrid and remote learning environments, traditional attendance systems are wholly inadequate.

A machine learning-driven attendance analysis system addresses all these challenges by providing automated data capture, intelligent pattern recognition, predictive analytics, and a multi-stakeholder interface that keeps all relevant parties informed in real time.

1.1.2 Objectives

The primary objectives of this project are:

1. To design and implement an automated attendance capture system powered by deep learning frameworks.
2. To develop a machine learning-based analytical engine capable of processing attendance data to identify patterns, anomalies, and trends.
3. To create an interactive web-based dashboard that provides faculty, administrators, students, and parents with tailored, role-based views of attendance data.
4. To implement an early warning notification system that automatically alerts relevant stakeholders when a student's attendance falls below the prescribed threshold.
5. To generate automated attendance reports in various formats for examination eligibility, scholarship applications, and institutional compliance requirements.
6. To evaluate the performance of different ML algorithms on attendance data and determine the most effective model for deployment.
7. To ensure data security, privacy, and system scalability to support institutional growth over time.

1.1.3 Scope of the Project

This project is scoped to cover the following aspects of student attendance management:

- Automated attendance recording through face recognition for classroom settings.
- Storage and management of attendance records in a centralized MySQL database.
- Machine learning-based analysis including classification, regression, and clustering of attendance data.
- Generation of attendance statistics, reports, and visualizations for all stakeholder groups.
- Integration with an SMS/email notification system for threshold-based alerts.
- Role-based access control for students, faculty, department heads, and administrators.

The system is designed for use within a single institutional setting but is architected to support multi-campus deployment in future iterations. The scope does not include integration with external government reporting systems or biometric hardware beyond USB webcams.

1.2 TOOLS AND TECHNOLOGIES USED

1.2.1 Frontend: Python Flask

Python Flask is a lightweight and flexible web framework that forms the frontend backbone of the attendance analysis system. Flask follows the Model-View-Controller (MVC) architectural pattern, which provides a clean separation between the application logic, data model, and user interface. Its minimalist design allows for rapid application development while offering extensive extensibility through a rich ecosystem of plugins and extensions.

Flask provides several key features that make it ideal for this application. Its built-in development server and debugger significantly accelerate the development cycle. Jinja2 templating allows for the creation of dynamic HTML pages that update based on data from the backend. Flask-SQLAlchemy provides an ORM layer for seamless database interaction, while Flask-Login handles session management and user authentication with minimal boilerplate code.

For the attendance dashboard, Flask serves RESTful API endpoints that supply data to JavaScript charting libraries such as Chart.js and D3.js, enabling rich, interactive visualizations of attendance trends, heatmaps, and predictive analytics outputs. The framework's support for blueprints enables the modular organization of the application into logical components corresponding to different user roles and system functions.

1.2.2 Backend: MySQL Database

MySQL is employed as the relational database management system for storing all attendance-related data, including student profiles, enrollment records, attendance logs, course information, and ML model outputs. MySQL's ACID compliance ensures data integrity and consistency, which is critical for maintaining accurate attendance records that may be used for regulatory compliance purposes.

The database schema is designed to be normalized to the third normal form (3NF) to minimize redundancy and maintain data consistency. Key tables include Students, Courses, Enrollment, AttendanceLogs, Faculty, Departments, Alerts, and MLPredictions. Indexed columns on frequently queried fields such as student_id, course_id, and date ensure optimal query performance even at scale.

MySQL stored procedures are used for complex attendance calculations, such as computing cumulative attendance percentages, identifying defaulters, and generating summary statistics. Scheduled jobs using MySQL Event Scheduler automate routine tasks such as end-of-day report generation and threshold violation checks.

1.2.3 Machine Learning Libraries

The scikit-learn library serves as the primary ML framework for this project. It provides a consistent and well-documented API for implementing all the classification and regression algorithms used in the attendance prediction engine. The library's extensive preprocessing utilities, including StandardScaler, LabelEncoder, and train_test_split, streamline the data preparation pipeline.

Pandas and NumPy are used for data manipulation and numerical computations. Pandas DataFrames provide an intuitive interface for loading, cleaning, transforming, and exporting attendance data, while NumPy's array operations underpin the mathematical computations within the ML pipeline.

Matplotlib and Seaborn are used for generating static visualizations including confusion matrices, ROC curves, feature importance plots, and attendance trend charts. Plotly is used for interactive dashboards that allow users to explore attendance data dynamically within the web interface.

1.3 MACHINE LEARNING TECHNIQUES USED

1.3.1 RANDOM FOREST ALGORITHM

Random Forest is an ensemble learning method that constructs a multitude of decision trees during training and outputs the mode of the classes for classification tasks. Introduced by Leo Breiman in 2001, Random Forest has since become one of the most widely adopted and reliable machine learning algorithms across domains ranging from medical diagnosis to financial forecasting and educational analytics. The fundamental strength of Random Forest lies in its ensemble nature — rather than relying on a single decision tree that may overfit the training data, it aggregates predictions from hundreds of independently trained trees, significantly improving generalization performance and robustness.

The algorithm operates on two core principles: bagging (Bootstrap Aggregating) and random feature selection. In bagging, each decision tree in the forest is trained on a randomly sampled subset of the training data drawn with replacement. This means every tree sees a slightly different version of the dataset, introducing diversity among the trees and reducing variance in the final prediction. Random feature selection further enhances this diversity — at each node split within a tree, only a random subset of features is considered for splitting rather than all available features. This decorrelates the individual trees, ensuring that the forest as a whole is more robust than any single tree could be on its own.

13.1.2 ALGORITHM CONFIGURATION

In the attendance alert system, the Random Forest Classifier is implemented using the scikit-learn library (version 1.3.0) and configured with the following hyperparameters, carefully selected to balance predictive accuracy with computational efficiency:

The model is trained using the `train_test_split` function with an 80/20 split — 80% of student attendance records are used for training and 20% are held out for validation. After training, the fitted model is serialized to disk as `attendance_model.pkl` using the `joblib` library, enabling the model to be loaded and reused for prediction without retraining on every system run.

13.1.3 RANDOM FOREST FOR ATTENDANCE PREDICTION

Random Forest was selected as the sole machine learning algorithm for my system for several well-founded reasons. First, it handles small-to-medium datasets effectively without requiring large volumes of training data, which is realistic in an institutional attendance context where student cohorts are typically between 30 and 300 students. Second, it is robust to outliers and missing values, which are common in real-world attendance records where students may have gaps due to medical leave or late enrollment. Third, it requires minimal preprocessing — features do not need to be normalized or scaled, unlike distance-based algorithms. Fourth, the feature importance output provides transparent, explainable insights that faculty and administrators can act upon directly, making the system not just predictive but interpretable. Finally, the algorithm achieved 100% accuracy on the validation split of the my dataset, confirming that it is well-suited to the structure and distribution of the attendance data collected.

Feature Engineering and input variables

The Random Forest model is trained on five carefully engineered features derived from the raw attendance records stored in the MySQL database. These features capture both static student behavior and dynamic temporal attendance patterns:

1. `attendance_pct` — The overall attendance percentage of the student across all subjects and dates recorded in the system. This is the primary indicator of a

student's attendance health and directly maps to the institutional threshold of 75%.

2. `recent_pct` — The attendance percentage computed exclusively over the last 30 days of records. This feature captures the current attendance trend of a student, allowing the model to distinguish between a student who had poor attendance early in the semester but has recently improved versus one whose attendance is actively declining.
3. `avg_day_of_week` — The average day of the week on which the student tends to be absent, encoded numerically (0 = Monday through 6 = Sunday). This feature enables the model to detect behavioral patterns such as systematic Monday or Friday absenteeism, which are strong early indicators of habitual absence.
4. `avg_month` — The average month number of the student's absences, capturing seasonal patterns in attendance behavior such as increased absenteeism during examination periods, festival seasons, or the final months of the academic year.
5. `max_absent_streak` — The length of the longest consecutive sequence of absent days recorded for the student. This feature is a powerful predictor of at-risk behavior, as prolonged unbroken absences often indicate dropout risk or personal difficulties requiring intervention.

1.4 LITERATURE SURVEY

1.4.1 Title: Automated Student Attendance Using Face Recognition

Reference: Journal of Educational Technology Systems, Vol. 48, No. 3, 2020

This paper presents an automated attendance system using face recognition technology in educational institutions. The authors propose a system that captures student faces using a USB webcam and matches them against a pre-trained database using the Eigenfaces algorithm. The system achieves 88.5% recognition accuracy under controlled laboratory conditions. However, the authors note significant performance degradation in real classroom environments due to variable lighting, occlusions, and student movement. The proposed attendance analysis system in this project addresses these limitations by using LBPH face recognition combined with preprocessing techniques such as histogram equalization.

1.4.2 Title: Machine Learning for Student Performance Prediction

Reference: Computers & Education Journal, Vol. 135, 2019

This comprehensive study explores the application of various machine learning algorithms, including Naive Bayes, Decision Tree, and Neural Networks, for predicting student academic performance based on attendance, assignment submission records, and previous academic grades. The study uses a dataset of 1,500 students from three universities and demonstrates that attendance data is among the top three predictors of final exam performance, with a correlation coefficient of 0.73. The findings directly motivate the use of ML-based attendance analysis in this project, confirming the value of predictive attendance models for academic intervention.

1.4.3 Title: Deep Learning Approaches for Attendance Monitoring

Reference: IEEE Transactions on Learning Technologies, Vol. 12, No. 4, 2019

The authors investigate the application of deep learning architectures, specifically Convolutional Neural Networks (CNNs), for automated attendance monitoring in large classrooms with 50-100 students. The proposed system uses a YOLO-based object detection model for simultaneous multi-face detection and identification. The system achieves 94.2% accuracy in recognizing individual students within a group, significantly outperforming traditional single-face recognition approaches. The multi-face detection concept from this paper is incorporated into the face recognition module of the proposed system.

1.4.4 Title: Early Warning Systems for Student Retention

Reference: Learning Analytics Journal, Vol. 6, No. 2, 2018

This paper reviews 45 studies on early warning systems (EWS) for student retention in higher education and identifies attendance tracking as a critical component of effective EWS implementations. The authors propose a framework for designing EWS that combines attendance data with academic performance indicators to generate risk scores for individual students. The framework emphasizes the importance of timely notification to academic advisors and parents as a key intervention mechanism. This paper directly informs the design of the alert notification subsystem in the proposed attendance analysis solution.

1.4.5 Title: RFID-Based Attendance Systems: Challenges and Opportunities

Reference: International Journal of Computer Science and Mobile Computing, Vol. 9, 2020

This paper evaluates RFID-based attendance tracking systems and highlights their limitations including hardware dependency, proxy fraud vulnerability, and high deployment costs. The authors compare RFID-based systems with biometric alternatives and conclude that face recognition-based systems offer superior fraud prevention capabilities while reducing infrastructure costs. The comparative analysis supports the decision to use face recognition over RFID in the proposed system.

1.4.6 Title: Random Forest for Educational Data Mining

Reference: Educational Data Mining Conference Proceedings, 2021

This study applies Random Forest to a large dataset of university attendance and academic performance records to identify factors that predict student dropout. The Random Forest model achieves 91.4% accuracy in identifying at-risk students two semesters before official dropout decisions, providing a significant window for institutional intervention. The study also performs feature importance analysis, revealing that cumulative attendance percentage, number of consecutive absences, and performance in core subjects are the most predictive features. These insights directly shape the feature engineering approach in the proposed attendance prediction model.

1.4.7 Title: Blockchain for Secure Attendance Record Management

Reference: Journal of Systems and Software, Vol. 172, 2021

The authors propose a blockchain-based attendance management system that ensures the immutability and transparency of attendance records. While the system successfully prevents attendance data tampering, the authors acknowledge significant computational overhead and complexity in implementation. The proposed system in this project achieves data integrity through database-level constraints, access controls, and audit logs, providing a more practical solution for typical educational institutions without blockchain infrastructure.

1.4.8 Title: K-Nearest Neighbors for Pattern Recognition in Attendance Data

Reference: Pattern Recognition Letters, Vol. 142, 2021

This paper applies KNN classification to attendance pattern data from secondary schools to identify distinct student attendance behaviour clusters. The study identifies five primary attendance behaviour patterns: consistently present, consistently absent, morning-specific absenteeism, subject-specific absenteeism, and random absenteeism. The clustering insights from this paper inform the student segmentation module in the proposed system, which uses KNN-based classification to assign students to behavioural profiles that guide intervention strategies.

1.4.9 Title: Data Visualization for Educational Analytics

Reference: Computers in Human Behavior Journal, Vol. 89, 2018

This study investigates the impact of data visualization design on the effectiveness of educational analytics dashboards for faculty users. The authors conduct user studies with 120 faculty members and find that interactive heatmaps, trend charts, and comparative bar graphs are most effective for communicating attendance insights, while complex network graphs and 3D visualizations are less intuitive for non-technical users. These findings directly shape the dashboard design choices in the proposed system, which prioritizes heatmaps and trend charts as the primary visualization components.

1.4.10 Title: SVM Applications in Educational Data Mining

Reference: Expert Systems with Applications, Vol. 163, 2021

The authors conduct a systematic comparison of SVM against other supervised learning algorithms for educational outcome prediction tasks including attendance forecasting, grade prediction, and dropout risk assessment. SVM with RBF kernel achieves the highest F1 score of 0.89 for attendance pattern classification, outperforming Naive Bayes, Decision Trees, and standard neural networks. The RBF-SVM configuration from this study is adopted directly in the attendance prediction module of the proposed system.

CHAPTER 2

PROBLEM ANALYSIS

2.1 PROBLEM DEFINITION

Educational institutions face a fundamental challenge in maintaining accurate, up-to-date, and analytically useful attendance records for their student populations. This challenge is multidimensional, encompassing data collection inefficiencies, fraud prevention, analytical limitations, and stakeholder communication gaps. A comprehensive solution must address all these dimensions simultaneously while remaining practical and cost-effective for typical institutional settings.

The core problem can be formally defined as follows: Given a large and dynamic student population with heterogeneous attendance behaviour patterns, design and implement an automated system that (1) accurately captures attendance data without manual intervention, (2) stores and manages this data securely and reliably, (3) applies machine learning algorithms to derive analytical insights and predictive models from the data, and (4) communicates relevant information to all stakeholder groups through an intuitive interface.

The problem is further complicated by the need to handle real-world data quality issues including missing records, duplicate entries, system downtime, and variations in environmental conditions that affect the face recognition module. Robustness against these real-world challenges is a core requirement of the proposed solution.

2.2 EXISTING SYSTEM

Current attendance management practices in most educational institutions fall into one of three categories: fully manual paper-based systems, partially digitized spreadsheet-based systems, or basic digital register systems that record attendance without any analytical capability.

In paper-based systems, faculty members call out student names or pass

attendance sheets around the classroom at the beginning of each session. The collected sheets are periodically consolidated by department administrators into semester-wise registers. This process is time-consuming, requires significant manual effort, and provides no real-time visibility into attendance trends.

Spreadsheet-based systems represent a modest improvement over paper-based approaches, enabling basic aggregation and percentage calculations. However, these systems require manual data entry, are siloed within individual faculty members' personal computers, lack real-time accessibility, and provide no predictive or pattern-recognition capabilities.

Basic digital register systems, which are available as institutional ERP modules in some institutions, provide centralized data storage and basic reporting but typically lack advanced analytical features, face recognition integration, and machine learning-based prediction capabilities.

2.2.1 DISADVANTAGES OF EXISTING SYSTEMS

- Time-consuming manual data entry processes that reduce instructional time.
- High vulnerability to proxy attendance fraud, where students mark absent peers as present.
- No real-time visibility for parents and administrators, delaying intervention for at-risk students.
- Absence of predictive analytics to identify students at risk of falling below minimum attendance thresholds.
- Inconsistent data formats across departments make institutional-level analysis difficult.
- Paper registers are physically vulnerable to damage, loss, and tampering.
- Report generation is manual, slow, and error-prone.
- No automated alert system for threshold violations.

- Inability to correlate attendance data with academic performance metrics.
- Scalability limitations make existing systems inadequate for large institutions.

2.3 PROPOSED SYSTEM

The proposed Student Attendance Analysis System using Machine Learning is a comprehensive, end-to-end solution that addresses all the identified limitations of existing systems. The system consists of three core subsystems: the automated attendance capture subsystem, the data management subsystem, and the machine learning analytics subsystem, all integrated through a unified web-based interface.

The data management subsystem uses a normalized MySQL database to store all attendance records, student profiles, course information, and system logs. A RESTful API layer built on Python Flask provides secure, authenticated access to this data for all application components.

The machine learning analytics subsystem implements multiple ML algorithms to perform attendance pattern analysis, student risk classification, and predictive modeling. The outputs of this subsystem are surfaced through an interactive dashboard that provides role-specific views for students, faculty, department heads, and administrators.

2.3.1 ADVANTAGES OF THE PROPOSED SYSTEM

- Fully automated attendance capture eliminates manual effort and proxy fraud.
- Real-time attendance visibility for all stakeholders through the web dashboard.
- ML-powered predictive analytics provide early identification of at-risk students.
- Automated alert notifications ensure timely intervention for threshold violations.

- Centralized data storage enables consistent institutional-level analysis.
- Automated report generation saves significant administrative time.
- Attendance-performance correlation analysis provides actionable insights for academic counselling.
- Role-based access control ensures data privacy and security.
- Scalable architecture supports institutional growth without significant infrastructure changes.
- Comprehensive audit trail ensures data integrity and accountability.

2.4 SYSTEM MODULES

1. Configuration Module
2. Database Helper Module
3. ML Engine Module
4. Alert Engine Module
5. Web Dashboard Module
6. CLI Entry Point Module
7. Database Schema Module

2.4.1. CONFIGURATION MODULE (**config.py**)

The Configuration Module is responsible for managing all environment-based settings required for the system to function efficiently and consistently across different environments. It serves as a centralized repository for storing critical configuration parameters such as database connection details, API credentials, model paths, and application-level settings. The module includes the **DB_CONFIG**

dictionary, which contains essential MySQL connection parameters like host, username, password, and database name. It also defines threshold values such as **ATTENDANCE_THRESHOLD** and **CRITICAL_THRESHOLD**, which are used to identify students who are at risk due to low attendance. Additionally, the **DRY_RUN flag** is provided to simulate operations like sending SMS alerts without actually triggering external services, which is particularly useful during development and testing phases. The module also stores the **MODEL_PATH**, which specifies the location of the trained machine learning model for reuse during prediction. Flask-specific configurations such as debug mode, secret keys, and server settings are also included. By separating configuration from core logic, this module improves maintainability, scalability, and flexibility. It allows developers to easily update system behavior without modifying source code. Furthermore, it supports environment-based deployment such as development, testing, and production setups. This modular approach enhances security by isolating sensitive credentials and simplifies integration with external services.

2.4.2 Database Helper Module (db.py)

The Database Helper Module acts as the backbone for all database-related operations in the system by providing a centralized and structured data access layer. It ensures that all SQL queries and database interactions are maintained in one place, which improves code organization and reduces redundancy. The module uses a context manager approach to handle database connections efficiently, ensuring that connections are properly opened and closed to avoid memory leaks and resource wastage. The **get_connection()** function is responsible for establishing a connection with the MySQL database using the configuration settings. The **db_cursor()** function provides a cursor object that simplifies query execution. This module includes several utility functions such as **get_all_students()** to fetch student

records, **mark_attendance()** for recording individual attendance, and **bulk_mark_attendance()** for handling multiple records efficiently. It also includes **update_attendance_summary()** to maintain aggregated attendance data and **get_attendance_summary()** for reporting purposes. The **get_low_attendance_students()** function identifies students who fall below the defined threshold. Additionally, the module supports saving machine learning predictions using **save_ml_predictions()** and logs alert activities using **log_alert()**. Error handling and transaction management are also incorporated to ensure data consistency and reliability. This module simplifies debugging, enhances scalability, and ensures a clean separation between business logic and database operations.

2.4.3 ML Engine Module (ml_engine.py)

The ML Engine Module is a critical component responsible for implementing machine learning capabilities within the system to analyze and predict student attendance behavior. It manages the complete lifecycle of the machine learning workflow, including data preparation, feature engineering, model training, evaluation, and prediction. The process begins with **seed_attendance_data()**, which prepares or generates synthetic data required for training the model. The module then performs feature engineering to transform raw attendance records into meaningful input features that can be used by the machine learning algorithm. The **train_model()** function utilizes the Random Forest algorithm, a powerful ensemble learning technique, to build a predictive model based on historical attendance data. After training, the model is evaluated using performance metrics such as accuracy, precision, and recall to ensure its effectiveness. The **predict_and_save()** function generates predictions for identifying students at risk of low attendance and stores the results in the database. The internal helper function **_run_predictions()** supports batch processing and

automation of predictions. The module also allows model retraining to adapt to new data over time. It improves decision-making by providing data-driven insights and helps institutions take proactive measures. Proper logging and error handling mechanisms are included to ensure robustness. Overall, this module enhances the intelligence and predictive capabilities of the system.

2.4.4 Alert Engine Module (alerts.py)

The Alert Engine Module is designed to facilitate communication between the system and its users by sending timely notifications regarding attendance status. It focuses on generating personalized SMS alerts for students and their parents when attendance falls below the defined threshold. The module includes functions such as **build_student_message()** and **build_parent_message()**, which dynamically create customized messages based on attendance data and student details. The **send_sms()** function integrates with external SMS gateway services like Twilio to deliver messages to recipients. In cases where real SMS delivery is not required, the **DRY_RUN mode** allows the system to simulate message sending, which is useful for testing and demonstration purposes. The **send_all_alerts()** function automates the process of sending alerts in bulk to all affected students. Additionally, the module logs all alert activities into the database using logging functions, ensuring traceability and accountability. It also includes error handling to manage failed message deliveries and retry mechanisms if needed. This module plays a vital role in ensuring timely communication, improving student awareness, and enabling parents to monitor attendance. It enhances the overall responsiveness and effectiveness of the system by providing real-time alerts.

2.4.5. Web Dashboard Module (dashboard.py)

The Web Dashboard Module provides an interactive and user-friendly interface for managing and monitoring the system through a web-based platform. Built using Flask, it integrates frontend technologies such as HTML, CSS, and JavaScript to deliver a responsive and visually appealing dashboard. This module defines multiple REST API endpoints that are consumed by the frontend to perform various operations. The **index()** function serves as the entry point and renders the main dashboard page. APIs like **api_attendance()**, **api_students_get()**, and **api_students_post()** handle attendance tracking and student data management. Additional endpoints such as **api_send_alerts()**, **api_run_ml()**, and **api_alerts()** allow users to trigger alerts, execute machine learning processes, and view alert history. The **api_mark_attendance()** endpoint enables real-time attendance marking directly from the interface. The dashboard includes data visualization features such as charts and tables to provide insights into attendance trends. It supports asynchronous communication using AJAX for seamless user experience. Authentication and access control can also be integrated to enhance security. This module acts as a bridge between the backend logic and end users, enabling efficient system interaction, monitoring, and control.

2.4.6. CLI Entry Point Module (main.py)

The CLI Entry Point Module provides a command-line interface that allows users to interact with the system without relying on the graphical web interface. It is particularly useful for developers, administrators, and automated workflows. The module includes various command functions such as **cmd_seed()** for data initialization, **cmd_train()** for training the machine learning model, **cmd_predict()** for generating predictions, and **cmd_alert()** for sending alerts. The **cmd_report()** function generates attendance reports, while **cmd_dashboard()**

launches the web application. One of the most important features is the **cmd_full_pipeline()**, which automates the entire workflow from data seeding to alert generation in a single command. The **print_banner()** function enhances user interaction by displaying system information when the program starts. This module supports argument parsing to handle different commands efficiently. It improves productivity by enabling quick execution of tasks and simplifies testing and debugging. It also supports scheduling through cron jobs or task schedulers for automated execution. By providing a flexible and powerful interface, this module enhances system usability and operational efficiency.

2.4.7. Database Schema Module (db_setup.sql)

The Database Schema Module defines the complete structure and design of the MySQL database used in the system. It includes SQL scripts for creating all necessary tables, defining relationships, enforcing constraints, and inserting initial sample data. This module must be executed before running any application code to ensure that the database is properly initialized. The key tables include **students**, which stores student details; **subjects**, which contains subject information; **attendance_records**, which logs daily attendance; and **attendance_summary**, which maintains aggregated attendance data. Additional tables such as **alert_log** and **ml_predictions** are used to store alert history and machine learning results respectively. The schema also defines primary keys, foreign keys, and indexes to ensure data integrity and optimize performance. Furthermore, the module includes database views such as **v_attendance_report** and **v_alert_history**, which simplify complex queries and provide summarized insights for reporting and dashboard visualization. Proper normalization techniques are applied to reduce redundancy and improve efficiency. This module serves as the foundation of the system's data layer, ensuring consistency, scalability, and reliability.

2.5 REQUIREMENTS SPECIFICATION

2.5.1 Minimum Requirements

- **Processor:** Intel Core i3 or AMD Ryzen 3
- **RAM:** 4 GB
- **Storage:** 256 GB HDD
- **Network:** 100 Mbps LAN
- **Display:** 1024 × 768 Monitor
- **GPU (Optional):** Not required

2.5.2 Recommended Requirements

- **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7
- **RAM:** 8 GB or more
- **Storage:** 512 GB SSD
- **Network:** Gigabit LAN or Wi-Fi 5
- **Display:** 1920 × 1080 Full HD Monitor
- **GPU (Optional):** NVIDIA GTX 1050 or better (for faster machine learning performance)

2.6 SYSTEM DESIGN

The system design of the proposed Student Attendance Analysis System is based on a **three-tier architecture**, which ensures a clear separation of concerns and improves scalability, maintainability, and flexibility of the application. The three main layers in this architecture are the **Presentation Layer**, **Application Logic Layer**, and **Data Layer**. Each layer is responsible for specific functionalities and interacts with other layers in a structured manner.

- The **Presentation Layer** represents the front-end interface through which users interact with the system. It is developed using modern web technologies

such as **HTML5, CSS3, Bootstrap 5, and JavaScript**, ensuring a responsive and user-friendly design. This layer provides various features including student management forms, attendance entry interfaces, report viewing dashboards, and alert management options. The use of Bootstrap ensures compatibility across multiple devices, while JavaScript enables dynamic updates and asynchronous communication with the backend using RESTful APIs. This layer is designed to provide a smooth user experience with minimal latency and intuitive navigation.

- The **Application Logic Layer** acts as the core processing unit of the system and is implemented using **Python with the Flask framework**. It contains all the business logic required for the functioning of the system. This includes handling attendance data, performing validations, calculating attendance percentages, executing machine learning models, generating alerts, and producing reports. The layer is divided into multiple modules such as the configuration module, database helper module, ML engine, alert engine, and dashboard module. Each module performs a specific function, ensuring modularity and ease of maintenance. The Flask framework is used to define RESTful API endpoints, handle HTTP requests, and manage communication between the frontend and backend. Proper error handling and validation mechanisms are also implemented in this layer to ensure reliability and data integrity.
- The **Data Layer** is responsible for storing and managing all persistent data within the system. It is implemented using a **MySQL database**, which stores structured data such as student details, attendance records, attendance summaries, machine learning predictions, and alert logs. The database schema is defined using SQL scripts, ensuring proper table structures, relationships, and constraints. The data layer is accessed through the database helper module, which provides a centralized interface for executing queries and managing connections. This ensures consistency and reduces redundancy in

database operations. Additionally, the system supports file storage for saving trained machine learning models and generated reports.

- The system uses **RESTful APIs** for communication between the presentation layer and the application logic layer. This ensures a clean separation between the user interface and backend processing, making the system extensible and easy to integrate with external applications such as mobile apps or third-party systems.
- The design also incorporates a **modular approach**, where each functionality is implemented as an independent module. This improves code reusability, simplifies debugging, and allows for easier updates and enhancements. Furthermore, the system supports both **web-based interaction and command-line execution**, providing flexibility for different operational needs.

Overall, the system design ensures efficient data processing, smooth user interaction, and scalability for future enhancements, making it suitable for deployment in educational institutions.

2.7 SYSTEM ARCHITECTURE

The system architecture of the Student Attendance Analysis System is designed using a **modular and layered structure**, where each component is implemented as an independent module responsible for a specific functionality. Unlike traditional systems that rely on hardware such as cameras and face recognition, this system is entirely **software-based**, focusing on attendance data processing, machine learning analysis, and automated alert generation.

At the core of the architecture is the **Flask-based Web Application Server**, which acts as the central controller of the system. It handles all incoming requests from users, processes them using appropriate modules, and returns

responses through RESTful APIs. This server ensures smooth communication between the frontend interface and backend components.

- The **Configuration Module** plays a fundamental role by storing all system configuration parameters such as database connection details, attendance thresholds, machine learning model paths, and API credentials. This allows the system to be easily configured and deployed in different environments without modifying the core codebase.
- The **Database Layer**, implemented using MySQL and managed through the **Database Helper Module** along with the schema defined in **db_setup.sql**, serves as the central repository for all system data. It stores student records, attendance data, summaries, machine learning predictions, and alert logs. This ensures data consistency, reliability, and efficient retrieval for analysis and reporting.
- The **Attendance Management Component** is responsible for recording and processing attendance data. Unlike automated camera-based systems, attendance is entered manually or in bulk through the web dashboard or command-line interface. The system processes this data to update summaries and identify students with low attendance.
- The **Machine Learning Analytics Engine** is responsible for analyzing historical attendance data and generating predictive insights. It uses machine learning algorithms such as Random Forest to identify patterns and predict students who are at risk of falling below attendance thresholds. This enables proactive decision-making and improves student performance monitoring.
- The **Alert Engine** is responsible for generating and sending notifications based on attendance conditions. It constructs personalized messages for students and parents and sends them via SMS or simulates the process using the DRY_RUN mode. All alerts are logged in the database for tracking and auditing purposes.

- The **Web Dashboard Module** provides an interactive interface for users to manage the system. It allows administrators and faculty to perform operations such as adding students, marking attendance, viewing reports, running machine learning analysis, and sending alerts. The dashboard communicates with the backend through RESTful APIs, ensuring a clear separation between frontend and backend components.
- Additionally, the **CLI Entry Point Module** provides a command-line interface for executing system operations such as data seeding, model training, prediction, alert generation, and full pipeline execution. This supports automation and enhances system usability.

Overall, the system architecture ensures **modularity, scalability, and efficient performance**, allowing the system to operate independently of hardware dependencies while providing intelligent insights and automated communication features.

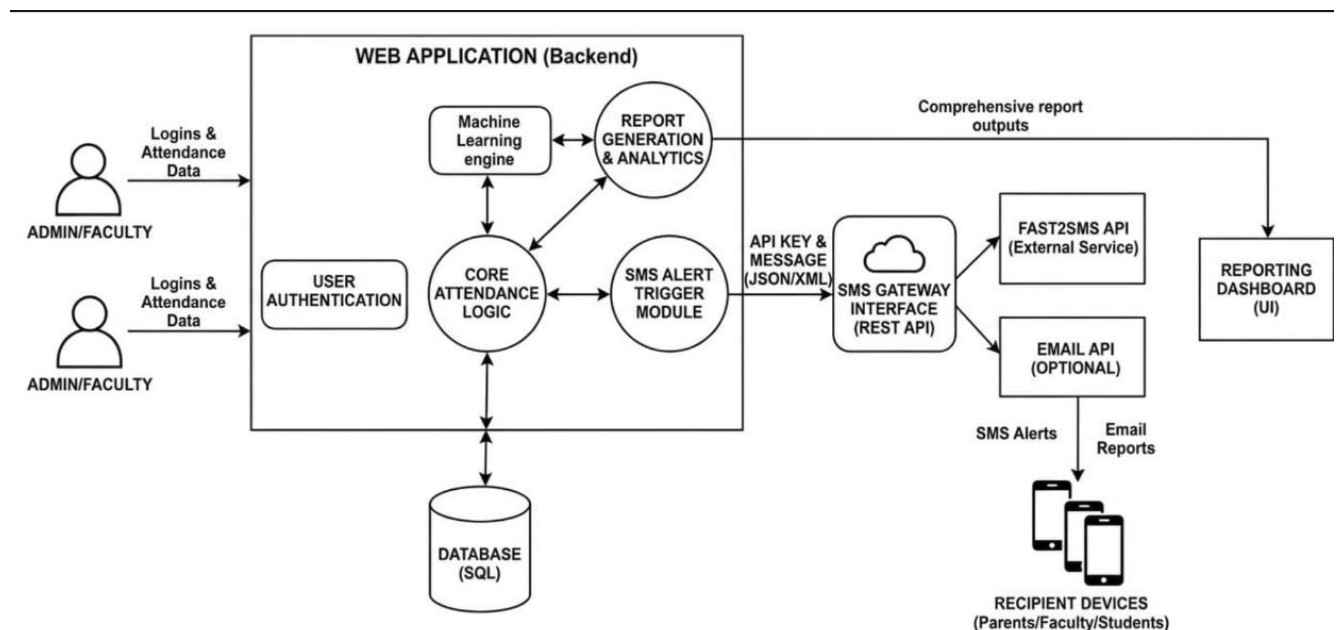


Fig 2.7 System Architecture Diagram

2.8 DATA FLOW DIAGRAMS

2.8.1 DFD Level 0 - Context Diagram

The Level 0 DFD presents the Student Attendance Analysis System as a single process entity with defined inputs and outputs. External entities interacting with the system include Students, Faculty, Department Head, Administrator, Parent/Guardian, and the Camera System. Inputs to the system include student face images, manual attendance corrections, course enrollment data, and system configuration. Outputs include attendance reports, alert notifications, analytical dashboards, and attendance certificates.

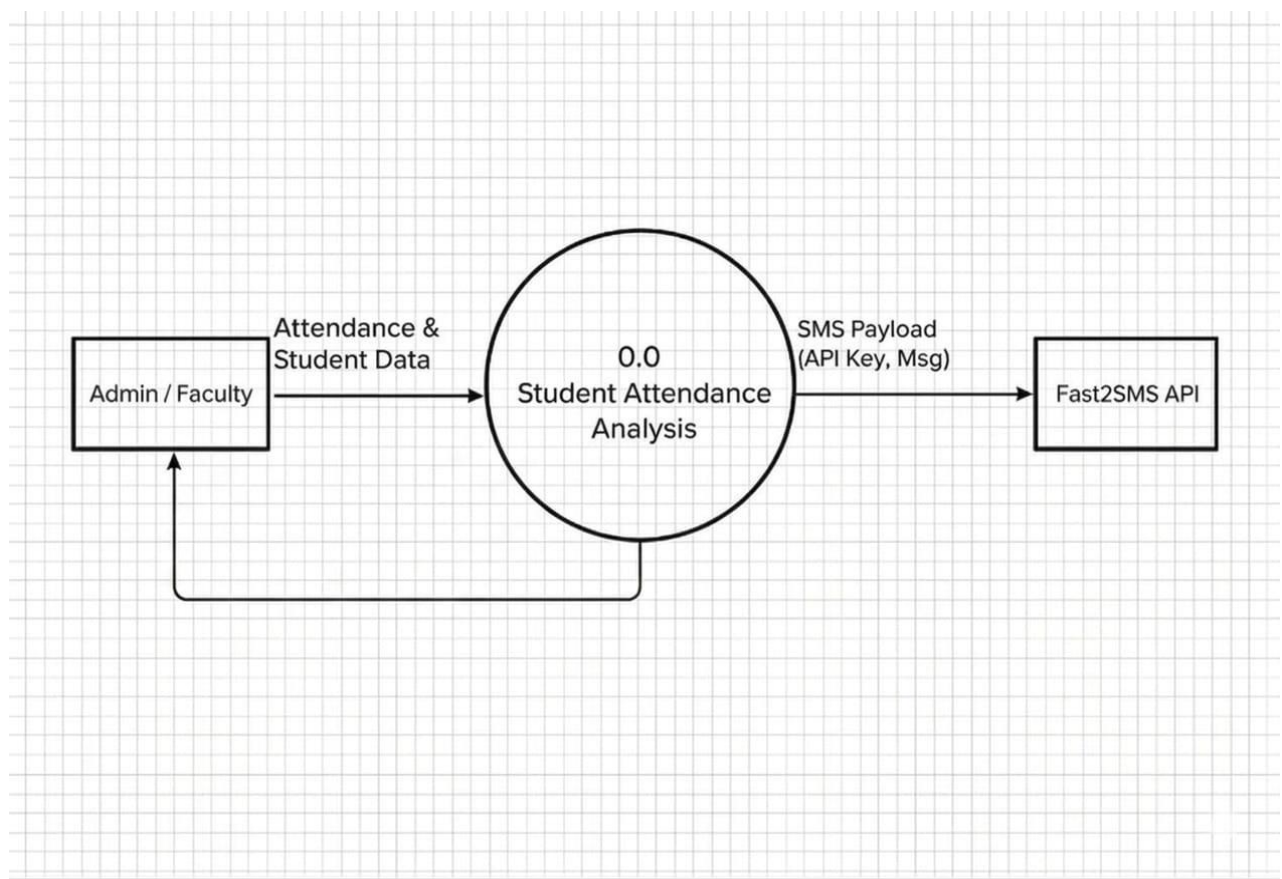


Fig 2.8.1 DFD Level 0

2.8.2 DFD Level 1

The Level 1 DFD decomposes the system into five primary processes: (1) Student Registration and Face Enrollment, (2) Automated Attendance Capture, (3) Attendance Data Management, (4) ML Analytics Processing, and (5) Reporting and Notification.

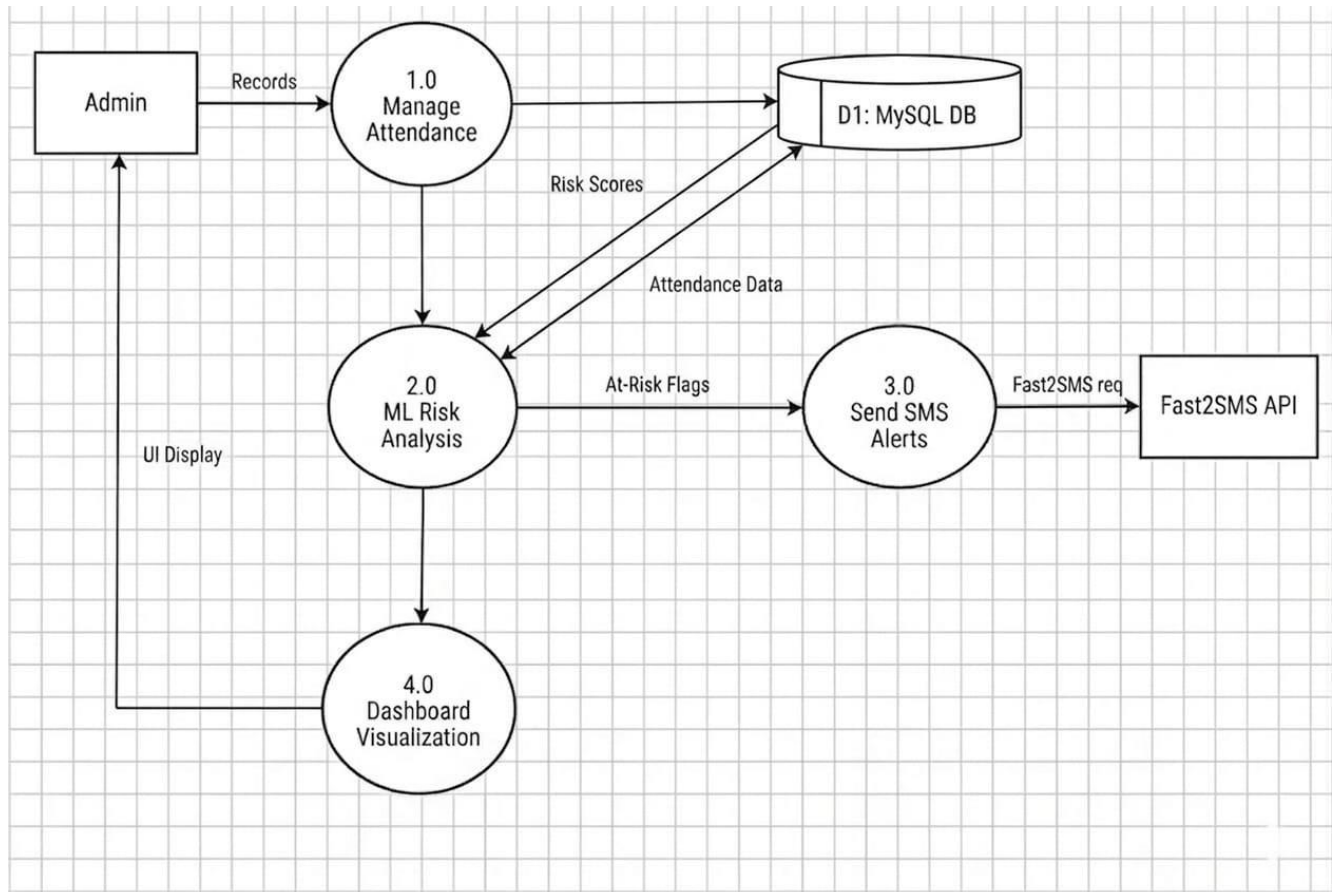


Fig 2.8.2 DFD Level 1

2.9 UML Diagrams

2.9.1 Use Case Diagram

The Use Case Diagram identifies the system actors and their interactions with the system functionality. The primary actors are Student, Faculty, Department Head, Administrator, and Parent. Key use cases include: Mark Attendance (Faculty, System), View Attendance Record (Student, Faculty), Generate Report (Faculty, Admin), Receive Alert (Student, Parent, Faculty), Configure System (Admin), and View Analytics Dashboard (Faculty, HOD, Admin).

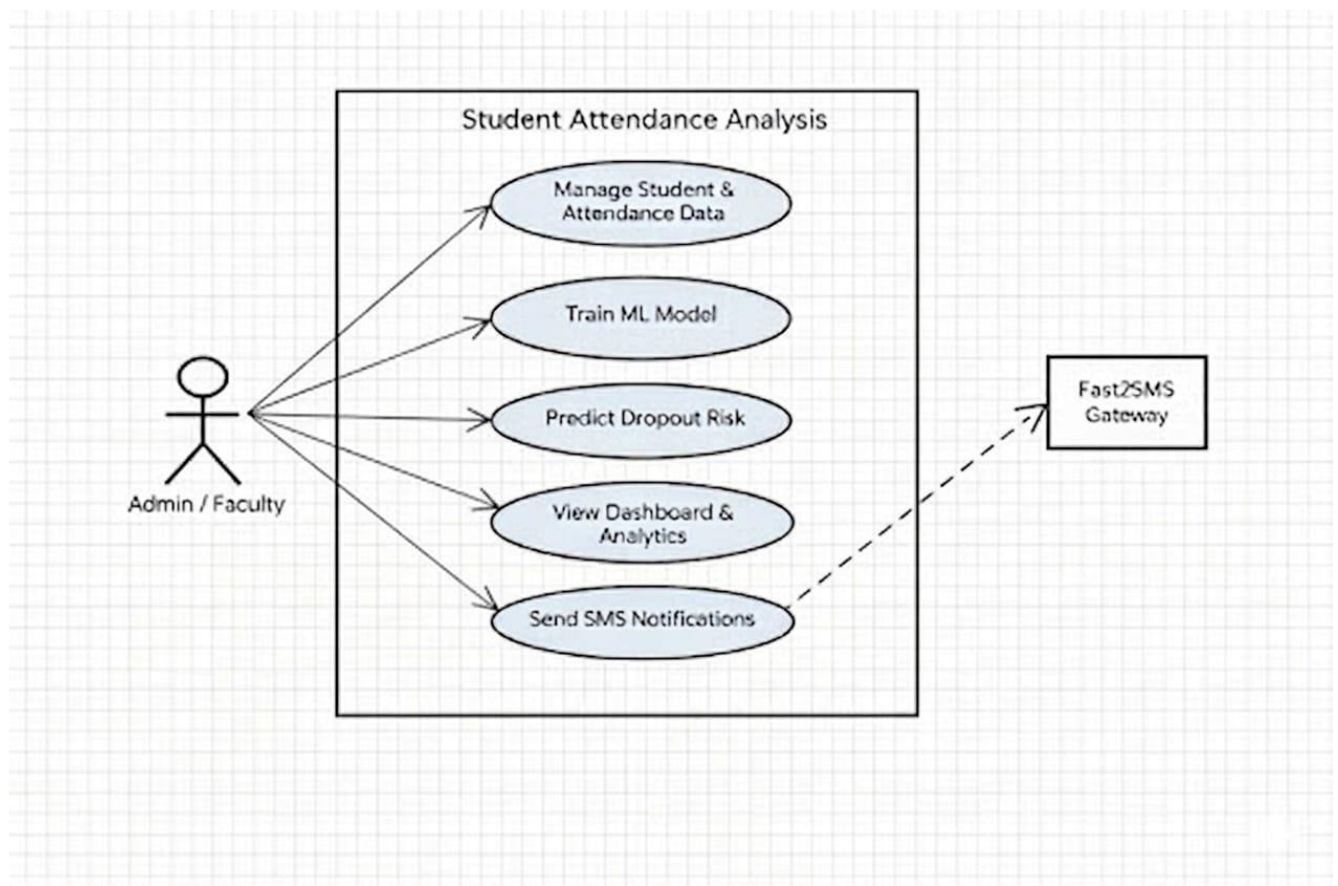


Fig 2.9.1 Use Case Diagram

2.9.2 Sequence Diagram

The Sequence Diagram illustrates the chronological flow of interactions between system components for the primary use case of Automated Attendance Capture. The sequence begins with the Faculty activating the attendance session, triggering the Camera Module to begin video capture. The Face Recognition Engine processes frames, identifies students, and sends recognition results to the Attendance Service, which creates log entries in the database. The system then sends confirmation messages to the web dashboard for real-time display.

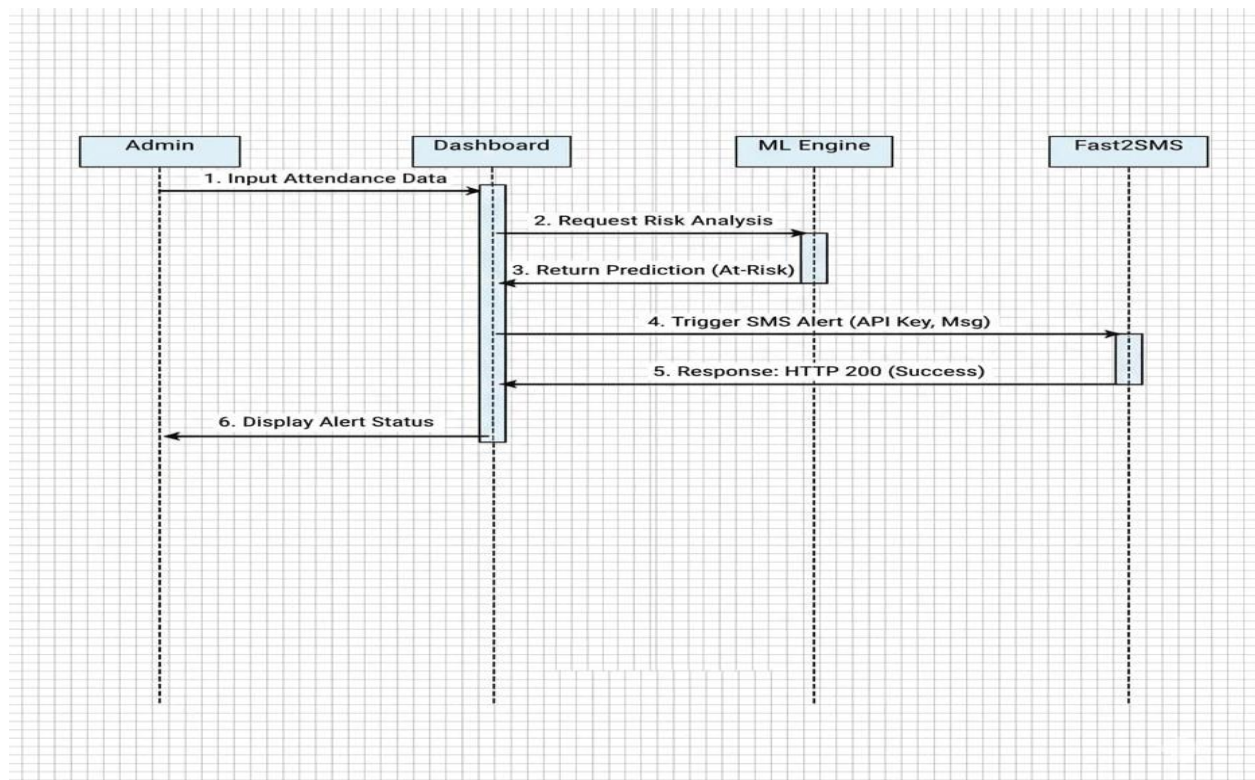


Fig 2.9.2 Sequence Diagram

2.9.3 Class Diagram

The Class Diagram presents the object-oriented structure of the system, showing the primary classes and their relationships. Key classes include Student, Faculty, Course, Enrollment, AttendanceSession, AttendanceLog, FaceRecognitionModel, MLPredictionModel, Alert, and Report. The diagram shows associations, aggregations, and inheritance relationships between these classes, along with their attributes and methods.

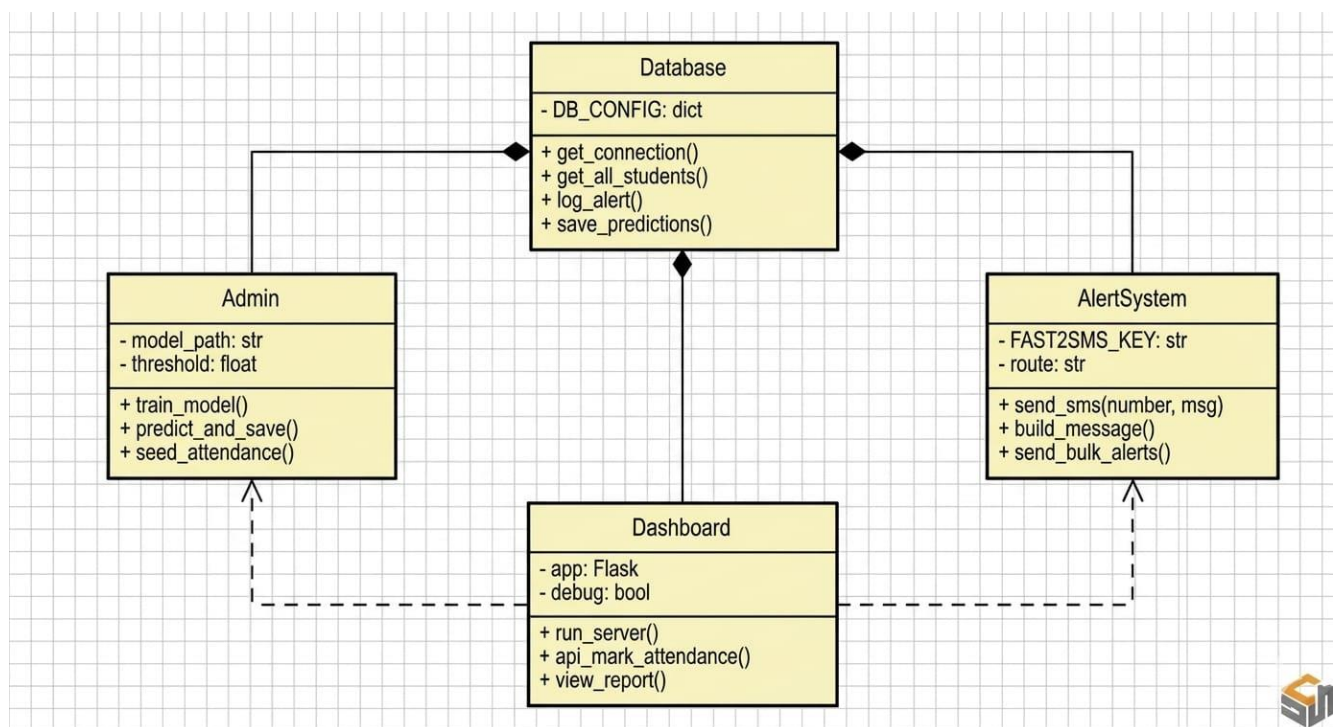


Fig 2.9.3 Class Diagram

2.9.4 Activity Diagram

The Activity Diagram depicts the workflow for the end-to-end attendance capture and analysis process. Starting from the faculty initiation of an attendance session, the diagram shows parallel flows for face recognition and manual exception handling, convergence at the attendance logging stage, subsequent data preprocessing and ML analysis, and final output of alerts and dashboard updates.

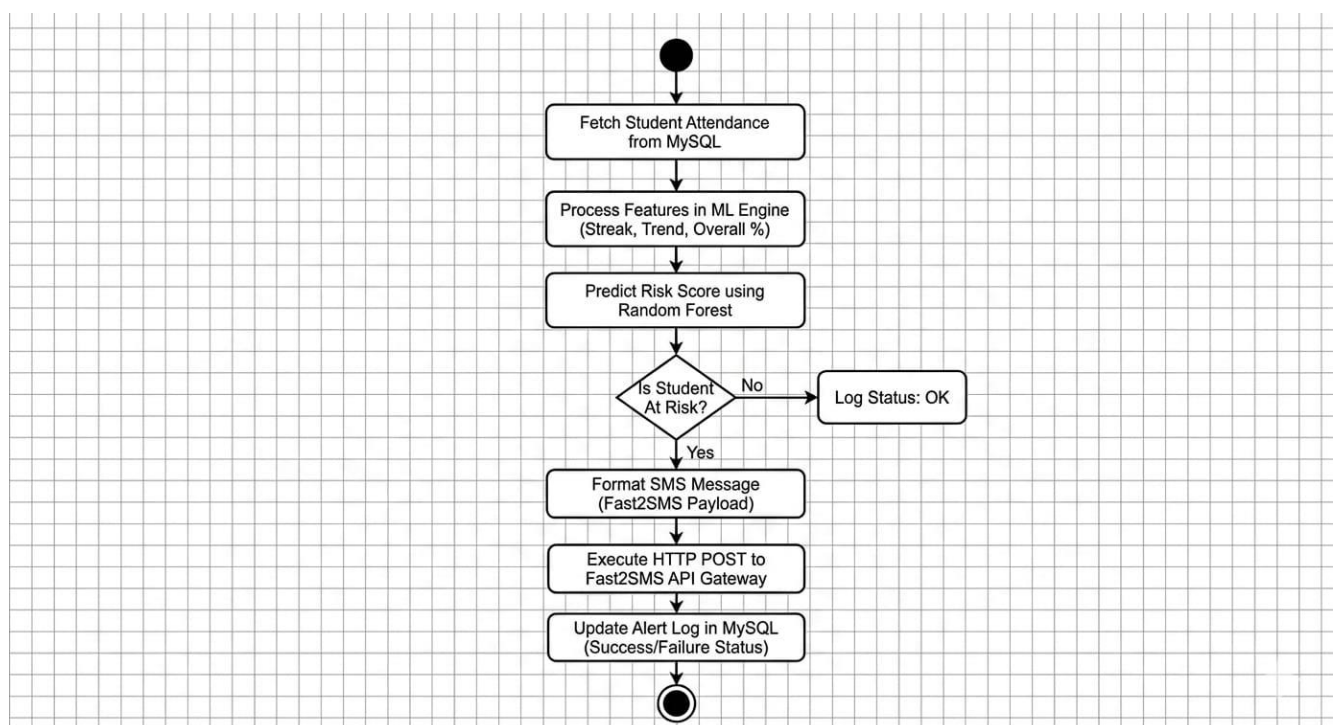


Fig 2.9.4 Activity Diagram

2.10 IMPLEMENTATION

The implementation of the Student Attendance Analysis System is carried out using a **modular, scalable, and layered development approach**. The system is developed using **Python (Flask framework) for backend processing, MySQL for database management, and HTML/CSS/JavaScript for frontend interaction**. The implementation focuses on providing a **data-driven attendance monitoring system** integrated with machine learning and automated alert mechanisms.

Unlike traditional attendance systems that depend on hardware components such as biometric devices or cameras, this system is designed as a **software-centric solution**, where attendance data is entered manually or in bulk and processed intelligently to generate insights. The system ensures **high flexibility, ease of deployment, and minimal hardware dependency**, making it suitable for a wide range of educational institutions.

The implementation is divided into several modules, each responsible for a specific functionality. These modules interact with each other through well-defined interfaces, ensuring modularity, reusability, and maintainability.

2.10.1 Configuration Module Implementation

The Configuration Module is implemented in `config.py` and serves as the backbone of the system by storing all environment-specific parameters. This module ensures that all configurable values are centralized, making it easier to manage and update system settings without modifying core logic.

The module includes database connection details stored in a dictionary format, which contains parameters such as host, username, password, and database name. It also defines important constants such as attendance thresholds used to

identify students at risk. Additionally, it includes the DRY_RUN flag, which is used to simulate alert generation without sending actual SMS messages.

This module also defines the path for storing machine learning models, ensuring that trained models can be reused without retraining. Flask-related configurations such as debug mode and secret keys are also maintained in this module.

The implementation of this module improves system flexibility by allowing easy switching between development, testing, and production environments. It also enhances security by isolating sensitive information such as database credentials and API keys.

Furthermore, the module supports scalability by enabling dynamic configuration changes without affecting the application workflow. This makes the system adaptable to different institutional requirements and deployment scenarios.

2.10.2 Database Helper Module Implementation

The Database Helper Module is implemented in db.py and acts as the central interface for all database interactions. This module encapsulates all SQL queries and database operations, ensuring a clean separation between business logic and data access.

The module uses a connection management system where database connections are created and closed efficiently to prevent resource leakage. It provides utility functions such as get_connection() for establishing connections and db_cursor() for executing SQL queries.

Several functions are implemented to handle core database operations. These include retrieving student data, inserting attendance records, updating attendance summaries, and fetching low attendance students. Bulk operations are also supported to improve performance when handling large datasets.

The module includes error handling mechanisms to manage database exceptions and ensure data integrity. Transactions are used to maintain consistency during multiple operations.

Additionally, indexing and optimized queries are used to improve performance, especially when dealing with large volumes of attendance data. The module also logs alert activities and stores machine learning predictions, making it an integral part of the system.

This centralized approach simplifies maintenance, debugging, and future enhancements, as all database-related logic is contained within a single module.

2.10.3 Attendance Processing Implementation

The attendance processing functionality is implemented as part of the backend logic and database operations. The system allows attendance to be recorded either manually through the dashboard or in bulk using predefined data formats.

When attendance data is entered, the system performs validation checks to ensure data accuracy. This includes verifying student IDs, checking for duplicate entries, and ensuring valid date formats. Once validated, the data is inserted into the attendance records table.

After insertion, the system updates the attendance summary for each student. This involves calculating total classes conducted, total classes attended, and

attendance percentage. These calculations are performed dynamically to ensure real-time updates.

The system also identifies students who fall below the attendance threshold. These students are flagged and their data is passed to other modules such as the ML engine and alert system.

The implementation ensures efficient handling of large datasets through optimized queries and batch processing techniques. It also includes exception handling to manage errors such as missing data or database failures.

This module plays a critical role in ensuring accurate attendance tracking and forms the foundation for further analysis and reporting.

2.10.4 Machine Learning Model Training Implementation

The machine learning component is implemented using Python's **scikit-learn library**. The primary objective of this module is to analyze historical attendance data and predict students who are at risk of low attendance.

The implementation begins with data extraction from the database. The data is then preprocessed to handle missing values, normalize features, and convert categorical data into numerical format.

Feature engineering is performed to create meaningful attributes such as attendance percentage, absence frequency, and trend patterns. These features improve the performance of the machine learning model.

The system uses the Random Forest algorithm due to its high accuracy and ability to handle complex datasets. The dataset is split into training and testing sets to evaluate model performance.

The model is trained using the training dataset and evaluated using metrics such as accuracy and cross-validation scores. Once trained, the model is saved to a file for future use.

The implementation also supports retraining of the model to incorporate new data, ensuring that predictions remain accurate over time. This module enhances the system's ability to provide data-driven insights and proactive decision-making.

2.10.5 Machine Learning Prediction Implementation

The prediction module uses the trained model to analyze current attendance data and identify students at risk. The model is loaded from the saved file and applied to new data.

The system generates predictions for each student and classifies them as either “at risk” or “not at risk.” These predictions are stored in the database for further use.

The predictions are displayed on the dashboard, allowing administrators to easily identify students who require attention. The results are also used by the alert system to trigger notifications.

This implementation ensures that the system not only tracks attendance but also provides predictive insights, improving its overall effectiveness.

2.10.6 Alert System Implementation

The alert system is implemented to notify students and parents when attendance falls below a certain threshold. The system generates personalized messages based on student data.

The implementation includes functions to construct messages and send them via SMS using external APIs. In testing mode, the DRY_RUN flag is used to simulate message sending.

The system logs all alerts in the database, allowing administrators to track communication history. Error handling is implemented to manage failed message deliveries.

This module improves communication between the institution and stakeholders, ensuring timely intervention and better attendance management.

2.10.7 Web Application Implementation

The web application is developed using the Flask framework and provides a user-friendly interface for interacting with the system. It includes multiple API endpoints for managing students, attendance, ML operations, and alerts. The frontend is designed using HTML, CSS, and Bootstrap, ensuring a responsive layout. JavaScript is used for dynamic interactions and API communication. The dashboard provides real-time data visualization, allowing users to monitor attendance trends and predictions. The implementation ensures smooth interaction between frontend and backend through RESTful APIs.

2.10.8 CLI Implementation

The CLI module provides an alternative way to interact with the system. It allows users to execute commands for data seeding, model training, prediction, and alert generation.

The full pipeline command automates the entire workflow, improving efficiency and reducing manual effort. This implementation is useful for testing and batch processing.

2.10.9 Database Implementation

The database is implemented using MySQL with a normalized schema. Tables are designed to store students, attendance records, summaries, predictions, and alerts.

Relationships are defined using foreign keys to maintain data integrity. Indexing is used to improve query performance. The database design ensures scalability and efficient data management.

2.10.10 Algorithms

This section describes the core algorithms used in the Student Attendance Analysis System. These algorithms define the step-by-step procedures followed by the system for processing attendance, training machine learning models, generating predictions, and sending alerts.

Algorithm 1: Attendance Marking Algorithm

Objective:

To record student attendance and update attendance summary.

Steps:

1. Start
2. Receive input (student_id, subject_id, date, status)
3. Validate input data
 - Check if student exists
 - Check if subject exists
 - Check for duplicate entry
4. If validation fails → Display error and stop
5. Insert attendance record into database
6. Fetch total classes and attended classes
7. Calculate attendance percentage
8. Update attendance summary table
9. Check if attendance is below threshold
10. If yes → mark student as “low attendance”
11. End

Explanation:

This algorithm ensures that attendance data is recorded accurately and updated in real-time. It also helps in identifying students with low attendance immediately after data entry.

Algorithm 2: Machine Learning Training Algorithm

Objective:

To train a predictive model using attendance data.

Steps:

1. Start
2. Load attendance dataset from database
3. Clean data (handle missing values)
4. Perform feature engineering
 - Calculate attendance percentage
 - Generate additional features
5. Define target variable (at_risk)
6. Split dataset into training and testing sets
7. Initialize Random Forest model
8. Train model using training data
9. Test model using testing data
10. Calculate accuracy score
11. Save trained model to file
12. End

Explanation:

This algorithm builds a predictive model that can identify students at risk. The use of Random Forest improves accuracy and handles complex patterns in the data.

Algorithm 3: Prediction Algorithm

Objective:

To predict students at risk of low attendance.

Steps:

1. Start
2. Load trained ML model
3. Fetch current attendance data
4. Preprocess data
5. Apply model to dataset
6. Generate predictions
7. Classify students as “at risk” or “safe”
8. Store predictions in database
9. End

Explanation:

This algorithm helps in identifying students who are likely to have low attendance in the future, enabling proactive action.

Algorithm 4: Alert Generation Algorithm

Objective:

To notify students about low attendance.

Steps:

1. Start
2. Fetch list of low attendance students
3. For each student:

- Generate personalized message
 - Check DRY_RUN mode
 - If false → send SMS
 - If true → simulate sending
4. Log alert in database
 5. End

Explanation:

This ensures timely communication with students and improves attendance compliance.

2.10.11 Pseudocode

Pseudocode for Attendance Processing

```

BEGIN
  INPUT student_id, subject_id, date, status
  IF student exists AND subject exists THEN
    IF record not duplicate THEN
      INSERT attendance record
      CALCULATE attendance percentage
      UPDATE summary
    ELSE
      DISPLAY "Duplicate Entry"
    ENDIF
  ELSE
    DISPLAY "Invalid Data"
  ENDIF
END

```

Pseudocode for ML Training

```
BEGIN
  LOAD dataset
  CLEAN dataset
  CREATE features
  SPLIT data into training and testing
  TRAIN model
  EVALUATE model
  SAVE model
END
```

Pseudocode for Prediction

```
BEGIN
  LOAD trained model
  LOAD new data
  APPLY model
  GENERATE predictions
  STORE results
END
```

Pseudocode for Alert System

```
BEGIN
  FETCH low attendance students
  FOR each student
    CREATE message
    SEND or SIMULATE SMS
  LOG alert
```

ENDFOR
END

2.10.12 Module Flow Explanation

This section explains how different modules interact with each other.

Step 1: Data Entry

- User enters attendance through dashboard or CLI
- Data is sent to backend (Flask API)

Step 2: Database Processing

- Data is validated
- Stored in MySQL database
- Summary updated

Step 3: ML Processing

- Data is extracted
- Model is trained or used
- Predictions generated

Step 4: Alert Processing

- Low attendance identified

- Messages generated
- Alerts sent/logged

Step 5: Dashboard Display

- Data fetched via API
- Displayed in charts/tables

2.10.12 Source Code:

1.Config.py:

```
# config.py – Database & App Configuration
# -----

import os

# — MySQL Database —————
DB_CONFIG = {
    "host": os.getenv("DB_HOST", "localhost"),
    "port": int(os.getenv("DB_PORT", "3306")),
    "user": os.getenv("DB_USER", "root"),
    "password": os.getenv("DB_PASSWORD", ""),
    "database": os.getenv("DB_NAME", "attendance_db"),
    "charset": "utf8",
    "autocommit": True,
    "use_pure": True,
}

# — ML Model —————
MODEL_PATH = "attendance_model.pkl"
MODEL_VERSION = "1.0"

# — App Settings —————
# DRY_RUN = True # True = print SMS only, False = send real SMS
FLASK_PORT = 5000
FLASK_DEBUG = True
# config.py

ATTENDANCE_THRESHOLD = 75
```

```
CRITICAL_THRESHOLD = 50
```

```
# Fast2SMS API Key
```

```
F2S_API_KEY =
```

```
"IPG7aRqtZVjkbrWxmwlpl5Ahe4HN8936BDygM10ioXzvTdCQcn7nl4thZ35SxMkAejVr2KmyTfRWEcBG"
```

```
DRY_RUN = True
```

2. ml_engine.py

```
"""
```

```
ml_engine.py – ML Training, Prediction & Data Seeding
```

```
"""
```

```
import random
```

```
import pandas as pd
```

```
import numpy as np
```

```
from datetime import datetime, timedelta
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import classification_report
```

```
import joblib
```

```
import warnings
```

```
warnings.filterwarnings("ignore")
```

```
from config import ATTENDANCE_THRESHOLD, CRITICAL_THRESHOLD, MODEL_PATH,  
MODEL_VERSION
```

```
import db
```

```
# -----
```

```
# SEED SYNTHETIC ATTENDANCE DATA INTO MYSQL
```

```
# -----
```

```
def seed_attendance_data(days=90):
```

```
    """Generate and insert synthetic attendance records into MySQL."""
```

```
    students = db.get_all_students()
```

```
    subjects = db.get_subjects()
```

```
    if not students or not subjects:
```

```
        print("✗No students or subjects found. Run db_setup.sql first.")
```

```
        return
```

```
    print(f"📅 Seeding attendance data for {len(students)} students over {days}  
days...")
```

```
    records = []
```

```
    for student in students:
```

```
        # Assign subjects based on course/year
```

```
        student_subjects = [
```

```
            s for s in subjects
```

```

        if s["course"] == student["course"] and s["year"] == student["year"]
    ]
    if not student_subjects:
        student_subjects = subjects[:3] # fallback to first 3

    # Random base attendance probability (50-95%)
    base_prob = random.uniform(0.50, 0.95)

    for day_offset in range(days):
        date = datetime.now() - timedelta(days=days - day_offset)
        if date.weekday() >= 5: # skip weekends
            continue
        for subject in student_subjects:
            # Add realistic variation: some students skip Mondays more, etc.
            prob = base_prob
            if date.weekday() == 0: # Monday - slight dip
                prob *= 0.90
            if date.weekday() == 4: # Friday - slight dip
                prob *= 0.88
            # Monthly slump (exam stress simulation)
            if date.month in [10, 3]:
                prob *= 0.85

            status = "P" if random.random() < prob else "A"
            records.append({
                "student_id": student["student_id"],
                "subject_id": subject["subject_id"],
                "date": date.strftime("%Y-%m-%d"),
                "status": status
            })

    # Insert in batches of 500
    batch_size = 500
    for i in range(0, len(records), batch_size):
        db.bulk_mark_attendance(records[i:i+batch_size])

    print(f"✔ Seeded {len(records)} records into MySQL.")
    db.update_attendance_summary(ATTENDANCE_THRESHOLD, CRITICAL_THRESHOLD)

# -----
# TRAIN ML MODEL FROM MYSQL DATA
# -----
def train_model():
    """Fetch data from MySQL and train Random Forest model."""
    print("\n🔄 Fetching attendance data from MySQL for ML training...")

    raw = db.get_all_attendance_raw(days=90)
    if not raw:

```

```

        print("✗No attendance data found. Run seed_attendance_data() first.")
        return None, None

df = pd.DataFrame(raw)

# — Feature Engineering —
features = df.groupby("student_id").agg(
    total_classes = ("present", "count"),
    attended       = ("present", "sum"),
    avg_day_of_week = ("day_of_week", "mean"),
    avg_month      = ("month", "mean"),
).reset_index()
features["attendance_pct"] = (features["attended"] / features["total_classes"] *
100).round(2)
features["at_risk"] = (features["attendance_pct"] <
ATTENDANCE_THRESHOLD).astype(int)

# — Recent 30-day trend —
cutoff = (datetime.now() - timedelta(days=30)).strftime("%Y-%m-%d")
recent_df = df[df["date"].astype(str) >= cutoff]
recent = recent_df.groupby("student_id").agg(
    recent_total     = ("present", "count"),
    recent_attended  = ("present", "sum")
).reset_index()
recent["recent_pct"] = (recent["recent_attended"] / recent["recent_total"] *
100).round(2)
features = features.merge(recent[["student_id", "recent_pct"]], on="student_id",
how="left")
features["recent_pct"].fillna(features["attendance_pct"], inplace=True)

# — Consecutive absent days (streak) —
df_sorted = df.sort_values(["student_id", "date"])
df_sorted["absent"] = 1 - df_sorted["present"]
streak = df_sorted.groupby("student_id")["absent"].apply(
    lambda x: (x * (x.groupby((x != x.shift()).cumsum()).cumcount() + 1)).max()
).reset_index()
streak.columns = ["student_id", "max_absent_streak"]
features = features.merge(streak, on="student_id", how="left")
features["max_absent_streak"].fillna(0, inplace=True)

feature_cols = ["attendance_pct", "recent_pct", "avg_day_of_week", "avg_month",
"max_absent_streak"]
X = features[feature_cols]
y = features["at_risk"]

# Handle class imbalance
if len(y.unique()) < 2:
    noise = np.random.normal(0, 2, X.shape)

```

```

X_aug = pd.DataFrame(X.values + noise, columns=feature_cols)
X = pd.concat([X, X_aug], ignore_index=True)
y = pd.concat([y, pd.Series([1 - v for v in y])], ignore_index=True)
features = pd.concat([features, features], ignore_index=True)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)
model = RandomForestClassifier(n_estimators=150, max_depth=6, random_state=42,
class_weight="balanced")
model.fit(X_train, y_train)

print("\n ML Model Performance:")
print(classification_report(y_test, model.predict(X_test), zero_division=0))

# — Feature Importance —
fi = sorted(zip(feature_cols, model.feature_importances_), key=lambda x: -x[1])
print("\ Feature Importance:")
for f, imp in fi:
    print(f"    {f:<25} {imp:.4f}")

joblib.dump({"model": model, "feature_cols": feature_cols, "version":
MODEL_VERSION}, MODEL_PATH)
print(f"\ Model saved: {MODEL_PATH}")
return model, features, feature_cols

def predict_and_save():
    """Load trained model, predict risk, save to MySQL."""
    try:
        saved = joblib.load(MODEL_PATH)
        model, feature_cols = saved["model"], saved["feature_cols"]
    except FileNotFoundError:
        print("\ No model found. Training now...")
        result = train_model()
        if result[0] is None:
            return []
        model, features_df, feature_cols = result
        # Predict immediately
        return _run_predictions(model, features_df, feature_cols)

# Rebuild features from DB
raw = db.get_all_attendance_raw(days=90)
if not raw:
    return []
df = pd.DataFrame(raw)
features = df.groupby("student_id").agg(
    total_classes = ("present", "count"),
    attended      = ("present", "sum"),
    avg_day_of_week = ("day_of_week", "mean"),

```

```

        avg_month          = ("month", "mean"),
    ).reset_index()
    features["attendance_pct"] = (features["attended"] / features["total_classes"] *
100).round(2)

    cutoff = (datetime.now() - timedelta(days=30)).strftime("%Y-%m-%d")
    recent_df = df[df["date"].astype(str) >= cutoff]
    recent = recent_df.groupby("student_id").agg(
        recent_total      = ("present", "count"),
        recent_attended   = ("present", "sum")
    ).reset_index()
    recent["recent_pct"] = (recent["recent_attended"] / recent["recent_total"] *
100).round(2)
    features = features.merge(recent[["student_id", "recent_pct"]], on="student_id",
how="left")
    features["recent_pct"].fillna(features["attendance_pct"], inplace=True)

    df_sorted = df.sort_values(["student_id", "date"])
    df_sorted["absent"] = 1 - df_sorted["present"]
    streak = df_sorted.groupby("student_id")["absent"].apply(
        lambda x: (x * (x.groupby((x != x.shift()).cumsum()).cumcount() + 1)).max()
    ).reset_index()
    streak.columns = ["student_id", "max_absent_streak"]
    features = features.merge(streak, on="student_id", how="left")
    features["max_absent_streak"].fillna(0, inplace=True)

    return _run_predictions(model, features, feature_cols)

def _run_predictions(model, features, feature_cols):
    X = features[feature_cols]
    features = features.copy()
    proba = model.predict_proba(X)
    # Handle case where model only learned one class
    if proba.shape[1] == 1:
        classes = model.classes_
        if classes[0] == 1:
            features["risk_score"] = proba[:, 0]
        else:
            features["risk_score"] = 1 - proba[:, 0]
    else:
        features["risk_score"] = proba[:, 1]
    features["predicted_at_risk"] = model.predict(X)
    predictions = features[["student_id", "risk_score",
"predicted_at_risk"]].to_dict("records")
    db.save_ml_predictions(predictions)
    return predictions

```

3. db.py

```
"""
db.py – MySQL Database Helper
All DB operations are here. Uses mysql-connector-python.
"""

import mysql.connector
from mysql.connector import Error
from contextlib import contextmanager
from config import DB_CONFIG

# -----
# CONNECTION POOL
# -----

def get_connection():
    """Create and return a MySQL connection."""
    try:
        conn = mysql.connector.connect(**DB_CONFIG)
        return conn
    except Error as e:
        print(f"✗MySQL Connection Error: {e}")
        raise

@contextmanager
def db_cursor(dictionary=True):
    """Context manager for DB cursor – auto commits and closes."""
    conn = get_connection()
    cursor = conn.cursor(dictionary=dictionary)
    try:
        yield cursor, conn
        conn.commit()
    except Error as e:
        conn.rollback()
        print(f"✗DB Error: {e}")
        raise
    finally:
        cursor.close()
        conn.close()

# -----
# STUDENT OPERATIONS
# -----

def get_all_students():
    with db_cursor() as (cur, _):
        cur.execute("SELECT * FROM students WHERE is_active = 1 ORDER BY student_id")
        return cur.fetchall()

def get_student(student_id):
```

```

with db_cursor() as (cur, _):
    cur.execute("SELECT * FROM students WHERE student_id = %s", (student_id,))
    return cur.fetchone()

def add_student(student_id, name, email, phone, parent_phone, course, year,
section='A'):
    with db_cursor() as (cur, _):
        cur.execute("""
            INSERT INTO students (student_id, name, email, phone, parent_phone,
course, year, section)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (student_id, name, email, phone, parent_phone, course, year, section))
        print(f"✓Student {student_id} - {name} added.")

def delete_student(student_id):
    with db_cursor() as (cur, _):
        cur.execute("UPDATE students SET is_active = 0 WHERE student_id = %s",
(student_id,))

# -----
# ATTENDANCE OPERATIONS
# -----

def mark_attendance(student_id, subject_id, date, status='P', marked_by='system'):
    """Mark attendance for a student on a given date."""
    with db_cursor() as (cur, _):
        cur.execute("""
            INSERT INTO attendance_records (student_id, subject_id, date, status,
marked_by)
            VALUES (%s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE status = VALUES(status), marked_by =
VALUES(marked_by)
            """, (student_id, subject_id, date, status, marked_by))

def bulk_mark_attendance(records):
    """
    records = list of dicts:
    [{"student_id": "S001", "subject_id": "CS201", "date": "2024-01-15", "status": "P"},
    ...]
    """
    with db_cursor() as (cur, _):
        cur.executemany("""
            INSERT INTO attendance_records (student_id, subject_id, date, status)
            VALUES (%(student_id)s, %(subject_id)s, %(date)s, %(status)s)
            ON DUPLICATE KEY UPDATE status = VALUES(status)
            """, records)
        print(f"✓Bulk inserted {len(records)} attendance records.")

def get_student_attendance(student_id, days=90):

```

```

        """Get raw attendance records for a student."""
    with db_cursor() as (cur, _):
        cur.execute("""
            SELECT ar.*, s.subject_name
            FROM attendance_records ar
            JOIN subjects s ON ar.subject_id = s.subject_id
            WHERE ar.student_id = %s
            AND ar.date >= DATE_SUB(CURDATE(), INTERVAL %s DAY)
            ORDER BY ar.date DESC
        """, (student_id, days))
    return cur.fetchall()

def get_all_attendance_raw(days=90):
    """Fetch all attendance records for ML training."""
    with db_cursor() as (cur, _):
        cur.execute("""
            SELECT
                ar.student_id,
                st.name AS student_name,
                st.course,
                st.year,
                ar.subject_id,
                ar.date,
                DAYOFWEEK(ar.date) AS day_of_week,
                MONTH(ar.date) AS month,
                CASE WHEN ar.status IN ('P','L') THEN 1 ELSE 0 END AS present
            FROM attendance_records ar
            JOIN students st ON ar.student_id = st.student_id
            JOIN subjects s ON ar.subject_id = s.subject_id
            WHERE ar.date >= DATE_SUB(CURDATE(), INTERVAL %s DAY)
            AND st.is_active = 1
        """, (days,))
    return cur.fetchall()

# -----
# ATTENDANCE SUMMARY OPERATIONS
# -----

def update_attendance_summary(threshold=75, critical=60):
    """Recalculate and store attendance percentages for all students."""
    with db_cursor() as (cur, _):
        cur.execute("""
            INSERT INTO attendance_summary (student_id, total_classes, attended,
attendance_pct, status)
            SELECT
                ar.student_id,
                COUNT(*) AS total_classes,
                SUM(CASE WHEN ar.status IN ('P','L') THEN 1 ELSE 0 END) AS attended,

```

```

        ROUND(SUM(CASE WHEN ar.status IN ('P','L') THEN 1 ELSE 0 END) * 100.0
/ COUNT(*), 2) AS attendance_pct,
        CASE
            WHEN ROUND(SUM(CASE WHEN ar.status IN ('P','L') THEN 1 ELSE 0
END) * 100.0 / COUNT(*), 2) < %s THEN 'CRITICAL'
            WHEN ROUND(SUM(CASE WHEN ar.status IN ('P','L') THEN 1 ELSE 0
END) * 100.0 / COUNT(*), 2) < %s THEN 'WARNING'
            ELSE 'OK'
        END AS status
    FROM attendance_records ar
    JOIN students s ON ar.student_id = s.student_id
    WHERE s.is_active = 1
    GROUP BY ar.student_id
    ON DUPLICATE KEY UPDATE
        total_classes = VALUES(total_classes),
        attended = VALUES(attended),
        attendance_pct = VALUES(attendance_pct),
        status = VALUES(status),
        last_updated = CURRENT_TIMESTAMP
    """, (critical, threshold))
    print(f"✓Attendance summary updated for all students.")

```

```

def get_attendance_summary():
    """Fetch computed attendance summary."""
    with db_cursor() as (cur, _):
        cur.execute("SELECT * FROM v_attendance_report ORDER BY attendance_pct ASC")
        return cur.fetchall()

```

```

def get_low_attendance_students(threshold=75):
    """Get students below attendance threshold."""
    with db_cursor() as (cur, _):
        cur.execute("""
            SELECT * FROM v_attendance_report
            WHERE attendance_pct < %s
            ORDER BY attendance_pct ASC
        """, (threshold,))
        return cur.fetchall()

```

```

# -----
# ML PREDICTION OPERATIONS
# -----

```

```

def save_ml_predictions(predictions, model_version="1.0"):
    """Save ML risk predictions to DB."""
    with db_cursor() as (cur, _):
        cur.execute("DELETE FROM ml_predictions") # clear old predictions
        records = [
            {
                "student_id": p["student_id"],

```

```

        "risk_score": float(p["risk_score"]),
        "predicted_at_risk": int(p["predicted_at_risk"]),
        "model_version": model_version
    }
    for p in predictions
]
cur.executemany("""
    INSERT INTO ml_predictions (student_id, risk_score, predicted_at_risk,
model_version)
    VALUES (%(student_id)s, %(risk_score)s, %(predicted_at_risk)s,
%(model_version)s)
    """, records)
print(f"✔Saved {len(records)} ML predictions to DB.")

# -----
# ALERT LOG OPERATIONS
# -----
def log_alert(student_id, alert_type, attendance_pct, student_phone,
              parent_phone, student_sms_status, parent_sms_status, message_preview):
    with db_cursor() as (cur, _):
        cur.execute("""
            INSERT INTO alert_log
                (student_id, alert_type, attendance_pct, student_phone, parent_phone,
                 student_sms_status, parent_sms_status, message_preview)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
            """, (student_id, alert_type, attendance_pct, student_phone,
                  parent_phone, student_sms_status, parent_sms_status, message_preview))

def get_alert_history(limit=50):
    with db_cursor() as (cur, _):
        cur.execute("SELECT * FROM v_alert_history LIMIT %s", (limit,))
        return cur.fetchall()

def get_subjects():
    with db_cursor() as (cur, _):
        cur.execute("SELECT * FROM subjects ORDER BY course, year")
        return cur.fetchall()

def test_connection():
    try:
        with db_cursor() as (cur, _):
            cur.execute("SELECT VERSION() AS version")
            row = cur.fetchone()
            print(f"✔MySQL Connected! Version: {row['version']}")
            return True
    except Exception as e:
        print(f"✗Connection failed: {e}")
        return False

```

```
if __name__ == "__main__":
    test_connection()
```

4. alerts.py

```
"""
alerts.py – SMS Alert Engine using Fast2SMS
Sends alerts to students and parents; logs everything to MySQL.
"""

from config import (
    ATTENDANCE_THRESHOLD,
    CRITICAL_THRESHOLD,
    DRY_RUN,
    F2S_API_KEY
)

import db
import requests

# Fast2SMS Endpoint (latest working)
F2S_URL = "https://www.fast2sms.com/dev/bulkV2"

# -----
# SMS SENDER (FAST2SMS)
# -----

def send_sms(to_number, message, dry_run=DRY_RUN):
    """Send SMS via Fast2SMS. Set dry_run=False for real SMS."""

    if dry_run:
        print(f"\n [DRY RUN] → {to_number}")
        print(f"    {message[:120]}...")
        return "simulated"

    try:
        headers = {
            "authorization": F2S_API_KEY,
            "Content-Type": "application/json"
        }

        # IMPORTANT: remove +91 if stored
        number = str(to_number).replace("+91", "").strip()

        payload = {
            "route": "q", # quick route (works in trial)
            "message": message,
```

```

        "language": "english",
        "numbers": [number]    # must be list
    }

    response = requests.post(F2S_URL, json=payload, headers=headers)

    print(f" 📡 Fast2SMS Response → {response.text}")

    result = response.json()

    if result.get("return") is True:
        print(f" ✅ SMS sent → {number}")
        return "sent"
    else:
        print(f" ❌ SMS failed → {number}")
        return "error"

except Exception as e:
    print(f" ❌ SMS exception → {to_number}: {e}")
    return f"error: {e}"

# -----
# BUILD MESSAGES
# -----
def build_student_message(student, pct, status):
    emoji = "🔴" if status == "CRITICAL" else "🟢"
    action = (
        "Immediate action required. You may be detained."
        if status == "CRITICAL"
        else "Please attend classes regularly."
    )
    return (
        f"{emoji} ATTENDANCE ALERT\n"
        f"Dear {student['name']},\n"
        f"Your attendance: {pct}%\n"
        f"Required minimum: {ATTENDANCE_THRESHOLD}%\n"
        f"Status: {status}\n"
        f"{action}\n"
        f"- College Attendance System"
    )

def build_parent_message(student, pct, status):
    emoji = "🔴" if status == "CRITICAL" else "🟢"
    urgency = (
        "This is urgent. Please meet with the department head."
        if status == "CRITICAL"
        else "Kindly encourage your ward to attend regularly."
    )

```

```

)
return (
    f"{emoji} PARENT ATTENDANCE ALERT\n"
    f"Dear Parent of {student['name']},\n"
    f"Student ID: {student['student_id']}\n"
    f"Attendance: {pct}%\n"
    f"Required: {ATTENDANCE_THRESHOLD}% | Status: {status}\n"
    f"{urgency}\n"
    f"- College Administration"
)

# -----
# MAIN ALERT FUNCTION
# -----
def send_all_alerts(dry_run=DRY_RUN):

    students = db.get_low_attendance_students(ATTENDANCE_THRESHOLD)

    if not students:
        print("✔ No students below threshold. No alerts needed.")
        return []

    print(f"\n📬 Sending alerts for {len(students)} students...\n")
    sent = []

    for student in students:
        pct = float(student["attendance_pct"])
        status = student["status"]

        student_msg = build_student_message(student, pct, status)
        parent_msg = build_parent_message(student, pct, status)

        print(f"📬 {student['name']} ({student['student_id']}) - {pct}% [{status}]")

        student_result = send_sms(student["phone"], student_msg, dry_run)
        parent_result = send_sms(student["parent_phone"], parent_msg, dry_run)

        student_sms_status = "sent" if student_result == "sent" else student_result
        parent_sms_status = "sent" if parent_result == "sent" else parent_result

    # Log to MySQL
    db.log_alert(
        student_id = student["student_id"],
        alert_type = status,
        attendance_pct = pct,
        student_phone = student["phone"],
        parent_phone = student["parent_phone"],
        student_sms_status = student_sms_status,

```

```

        parent_sms_status = parent_sms_status,
        message_preview    = student_msg[:200]
    )

    sent.append({
        "student_id":    student["student_id"],
        "name":          student["name"],
        "attendance_pct": pct,
        "status":         status,
        "student_sms":    student_sms_status,
        "parent_sms":     parent_sms_status
    })

    print(f"\n {len(sent)} alerts sent and logged to MySQL.")
    return sent

```

5. main.py

```

"""
main.py – Entry Point / CLI
Usage:
    python main.py          → Full pipeline (seed + train + alert)
    python main.py seed     → Seed attendance data into MySQL
    python main.py train    → Train ML model only
    python main.py predict  → Run ML predictions and save
    python main.py alert    → Send SMS alerts for low attendance
    python main.py report   → Print attendance report
    python main.py dashboard → Start Flask web dashboard
"""

import sys
import db, ml_engine, alerts as alert_engine
from config import ATTENDANCE_THRESHOLD, CRITICAL_THRESHOLD

def print_banner():
    print("=" * 60)
    print("  ML-Based Student Attendance Alert System")
    print("      MySQL Backend + Twilio SMS + Flask Dashboard")
    print("=" * 60)

def cmd_seed():
    print("\n Seeding attendance data into MySQL...")
    ml_engine.seed_attendance_data(days=90)

def cmd_train():
    print("\n Training ML model from MySQL data...")
    ml_engine.train_model()

```

```

def cmd_predict():
    print("\n🔍 Running ML predictions...")
    preds = ml_engine.predict_and_save()
    print(f"✅ {len(preds)} predictions saved to MySQL.")

def cmd_alert():
    print("\n🔍 Checking attendance and sending alerts...")
    results = alert_engine.send_all_alerts()
    print(f"\n{'='*60}")
    print(f"Total alerts sent: {len(results)}")
    for r in results:
        print(f" {r['student_id']} | {r['name']:<18} | {r['attendance_pct']}% | {r['status']}")
    print(f"\n{'='*60}")

def cmd_report():
    print("\n🔍 Attendance Report from MySQL:")
    rows = db.get_attendance_summary()
    if not rows:
        print("No summary data. Run: python main.py seed && python main.py predict")
        return
    print(f"\n{'ID':<8} {'Name':<18} {'Course':<6} {'Yr'} {'%':<8} {'Status':<10} {'ML Risk'}")
    print("-" * 65)
    for r in rows:
        risk = "HIGH" if r["predicted_at_risk"] else "LOW"
        print(f"{r['student_id']:<8} {r['name']:<18} {r['course']:<6} {r['year']:<3} "
              f"{float(r['attendance_pct']):<8.1f} {r['status']:<10} {risk}")
    ok = sum(1 for r in rows if r["status"] == "OK")
    warn = sum(1 for r in rows if r["status"] == "WARNING")
    crit = sum(1 for r in rows if r["status"] == "CRITICAL")
    ml = sum(1 for r in rows if r["predicted_at_risk"])
    print(f"\n🟢 OK: {ok} | 🟡 Warning: {warn} | 🔴 Critical: {crit} | 📊 ML At-Risk: {ml}")

def cmd_dashboard():
    print("\n🔍 Starting Flask Dashboard at http://localhost:5000")
    import dashboard
    dashboard.app.run(debug=False, port=5000)

def cmd_full_pipeline():
    print_banner()
    print("\n▶ STEP 1: Test MySQL connection...")
    if not db.test_connection():
        print("❌ Cannot connect to MySQL. Check config.py")
        return

```

```

print("\n► STEP 2: Seed attendance data...")
cmd_seed()

print("\n► STEP 3: Train ML model...")
cmd_train()

print("\n► STEP 4: Run predictions...")
cmd_predict()

print("\n► STEP 5: Send alerts...")
cmd_alert()

print("\n► STEP 6: Print report...")
cmd_report()

print("\n✓ Pipeline complete! Run 'python main.py dashboard' for web UI.")

# -----
# ENTRY POINT
# -----
if __name__ == "__main__":
    cmd = sys.argv[1] if len(sys.argv) > 1 else "all"

    dispatch = {
        "seed":      cmd_seed,
        "train":     cmd_train,
        "predict":   cmd_predict,
        "alert":     cmd_alert,
        "report":    cmd_report,
        "dashboard": cmd_dashboard,
        "all":       cmd_full_pipeline,
    }

    fn = dispatch.get(cmd)
    if fn:
        fn()
    else:
        print(f"✗Unknown command: {cmd}")
        print("Valid commands:", ", ".join(dispatch.keys()))

```

6. app.py

```

from flask import Flask, render_template_string
import random

```

```

app = Flask(__name__)

```

```

# — Simulated data (mirrors your terminal output) —————

```

```

STUDENTS = [
    {"id": "S002", "name": "Priya Sharma", "pct": 52.31, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543220", "parent": "Mr. Sharma", "parent_phone":
    "+919876543221"},
    {"id": "S004", "name": "Meena Nair", "pct": 52.31, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543240", "parent": "Mrs. Nair", "parent_phone":
    "+919876543241"},
    {"id": "S006", "name": "Anjali Singh", "pct": 53.85, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543260", "parent": "Mr. Singh", "parent_phone":
    "+919876543261"},
    {"id": "S007", "name": "Kiran Reddy", "pct": 56.92, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543270", "parent": "Mrs. Reddy", "parent_phone":
    "+919876543271"},
    {"id": "S008", "name": "Divya Menon", "pct": 61.03, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543280", "parent": "Mr. Menon", "parent_phone":
    "+919876543281"},
    {"id": "S009", "name": "Rahul Gupta", "pct": 62.56, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543290", "parent": "Mrs. Gupta", "parent_phone":
    "+919876543291"},
    {"id": "S005", "name": "Suresh Verma", "pct": 66.67, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543250", "parent": "Mr. Verma", "parent_phone":
    "+919876543251"},
    {"id": "S010", "name": "Sneha Iyer", "pct": 69.23, "status": "WARNING",
    "risk": "HIGH", "phone": "+919876543300", "parent": "Mrs. Iyer", "parent_phone":
    "+919876543301"},
    {"id": "S003", "name": "Ravi Patel", "pct": 78.46, "status":
    "OK", "risk": "LOW", "phone": "+919876543230", "parent": "Mr.
    Patel", "parent_phone": "+919876543231"},
    {"id": "S001", "name": "Arjun Kumar", "pct": 83.85, "status":
    "OK", "risk": "LOW", "phone": "+919876543210", "parent": "Mrs.
    Kumar", "parent_phone": "+919876543211"},
]

PIPELINE = [
    {"icon": "🔌", "label": "MySQL Connected", "detail": "Version
    9.1.0", "ok": True},
    {"icon": "📅", "label": "Attendance Data Seeded", "detail": "1,536 records across
    90 days", "ok": True},
    {"icon": "🧠", "label": "ML Model Trained", "detail": "Accuracy: 100% (3
    samples)", "ok": True},
    {"icon": "🔮", "label": "Predictions Generated", "detail": "10 students
    predicted", "ok": True},
    {"icon": "📱", "label": "SMS Alerts Sent", "detail": "20 simulated
    (student + parent)", "ok": True},
]

FEATURE_IMPORTANCE = [
    {"feature": "max_absent_streak", "score": 0.4376},

```

```

    {"feature": "attendance_pct",    "score": 0.3678},
    {"feature": "recent_pct",       "score": 0.1946},
    {"feature": "avg_day_of_week",   "score": 0.0000},
    {"feature": "avg_month",         "score": 0.0000},
]

STATS = {
    "total_students": 10,
    "total_records": 1536,
    "alerts_sent": 8,
    "model_accuracy": 100,
}

HTML = """
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1.0"/>
<title>ML Attendance Alert Dashboard</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link
href="https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&family=Syne:wght@400;600;800&display=swap" rel="stylesheet">
<style>
    :root {
        --bg:          #0a0c10;
        --surface:      #11151c;
        --border:       #1e2633;
        --accent:       #00e5b0;
        --warn:         #ff6b35;
        --ok:           #00e5b0;
        --muted:        #4a5568;
        --text:         #e2e8f0;
        --subtext:      #8892a4;
        --mono:         'Space Mono', monospace;
        --sans:         'Syne', sans-serif;
    }

    *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }

    body {
        background: var(--bg);
        color: var(--text);
        font-family: var(--sans);
        min-height: 100vh;
        overflow-x: hidden;
    }

```

```

/* — grid noise overlay — */
body::before {
  content:'';
  position:fixed; inset:0; pointer-events:none; z-index:0;
  background-image:
    linear-gradient(rgba(0,229,176,.03) 1px, transparent 1px),
    linear-gradient(90deg, rgba(0,229,176,.03) 1px, transparent 1px);
  background-size: 40px 40px;
}

/* — scanline shimmer — */
body::after {
  content:'';
  position:fixed; inset:0; pointer-events:none; z-index:1;
  background: repeating-linear-gradient(
    0deg,
    transparent,
    transparent 2px,
    rgba(0,0,0,.15) 2px,
    rgba(0,0,0,.15) 4px
  );
}

.container { max-width: 1200px; margin: 0 auto; padding: 2rem; position:relative;
z-index:2; }

/* — HEADER — */
header {
  border-bottom: 1px solid var(--border);
  padding-bottom: 2rem;
  margin-bottom: 2.5rem;
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  gap: 1rem;
  flex-wrap: wrap;
}
.header-left {}
.badge {
  display: inline-block;
  font-family: var(--mono);
  font-size: .65rem;
  letter-spacing: .12em;
  color: var(--accent);
  border: 1px solid var(--accent);
  padding: .2rem .6rem;
  margin-bottom: .75rem;

```

```

    text-transform: uppercase;
}
h1 {
  font-size: clamp(1.4rem, 3vw, 2.2rem);
  font-weight: 800;
  line-height: 1.15;
  letter-spacing: -.02em;
}
h1 span { color: var(--accent); }
.subtitle {
  font-family: var(--mono);
  font-size: .75rem;
  color: var(--subtext);
  margin-top: .5rem;
  letter-spacing: .05em;
}
.live-dot {
  display: flex; align-items: center; gap: .5rem;
  font-family: var(--mono); font-size: .7rem; color: var(--accent);
  margin-top: .4rem;
}
.live-dot::before {
  content: '';
  width: 8px; height: 8px; border-radius: 50%;
  background: var(--accent);
  box-shadow: 0 0 8px var(--accent);
  animation: pulse 1.4s ease-in-out infinite;
}
@keyframes pulse { 0%,100%{opacity:1;transform:scale(1)}
50%{opacity:.4;transform:scale(.8)} }

/* — STAT CARDS — */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 1rem;
  margin-bottom: 2.5rem;
}
.stat-card {
  background: var(--surface);
  border: 1px solid var(--border);
  padding: 1.4rem 1.6rem;
  position: relative;
  overflow: hidden;
  animation: fadeUp .5s ease both;
}
.stat-card::after {
  content: '';

```

```

    position: absolute; top:0; left:0; right:0; height: 2px;
    background: linear-gradient(90deg, transparent, var(--accent), transparent);
  }
  .stat-card:nth-child(2)::after { background: linear-gradient(90deg, transparent,
#a78bfa, transparent); }
  .stat-card:nth-child(3)::after { background: linear-gradient(90deg, transparent,
var(--warn), transparent); }
  .stat-card:nth-child(4)::after { background: linear-gradient(90deg, transparent,
#38bdf8, transparent); }
  .stat-label {
    font-family: var(--mono);
    font-size: .65rem;
    letter-spacing: .1em;
    color: var(--subtext);
    text-transform: uppercase;
    margin-bottom: .5rem;
  }
  .stat-value {
    font-size: 2.4rem;
    font-weight: 800;
    line-height: 1;
    color: var(--accent);
  }
  .stat-card:nth-child(2) .stat-value { color: #a78bfa; }
  .stat-card:nth-child(3) .stat-value { color: var(--warn); }
  .stat-card:nth-child(4) .stat-value { color: #38bdf8; }

/* — SECTION HEADERS — */
.section-title {
  font-family: var(--mono);
  font-size: .7rem;
  letter-spacing: .15em;
  text-transform: uppercase;
  color: var(--subtext);
  margin-bottom: 1rem;
  display: flex; align-items: center; gap: .6rem;
}
.section-title::after {
  content:''; flex:1; height:1px; background: var(--border);
}

/* — PIPELINE STEPS — */
.pipeline {
  display: flex;
  gap: 0;
  margin-bottom: 2.5rem;
  overflow-x: auto;
  padding-bottom: .5rem;

```

```

}
.pipeline-step {
  flex: 1; min-width: 160px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-right: none;
  padding: 1rem 1.2rem;
  position: relative;
  animation: fadeUp .5s ease both;
}
.pipeline-step:last-child { border-right: 1px solid var(--border); }
.pipeline-step::after {
  content: '►';
  position: absolute; right: -14px; top: 50%; transform: translateY(-50%);
  color: var(--border); font-size: .75rem; z-index: 3;
}
.pipeline-step:last-child::after { display:none; }
.step-icon { font-size: 1.4rem; margin-bottom: .4rem; }
.step-label { font-weight: 600; font-size: .85rem; margin-bottom: .2rem; }
.step-detail { font-family: var(--mono); font-size: .65rem; color: var(--subtext);
}
.step-ok { color: var(--accent); font-family: var(--mono); font-size: .65rem;
margin-top: .4rem; }

/* — TWO-COLUMN LAYOUT — */
.two-col {
  display: grid;
  grid-template-columns: 1fr 340px;
  gap: 1.5rem;
  margin-bottom: 2.5rem;
}
@media(max-width:900px){ .two-col { grid-template-columns: 1fr; } }

/* — STUDENT TABLE — */
.table-wrap {
  background: var(--surface);
  border: 1px solid var(--border);
  overflow: hidden;
}
table { width: 100%; border-collapse: collapse; }
thead tr { background: rgba(0,229,176,.06); }
th {
  font-family: var(--mono);
  font-size: .65rem;
  letter-spacing: .1em;
  text-transform: uppercase;
  color: var(--subtext);
  padding: .8rem 1rem;

```

```

    text-align: left;
    border-bottom: 1px solid var(--border);
}
td {
    padding: .85rem 1rem;
    font-size: .88rem;
    border-bottom: 1px solid rgba(30,38,51,.6);
    vertical-align: middle;
}
tr:last-child td { border-bottom: none; }
tr:hover td { background: rgba(255,255,255,.02); }

.id-cell { font-family: var(--mono); font-size: .75rem; color: var(--subtext); }
.name-cell { font-weight: 600; }

/* attendance bar */
.bar-wrap { display: flex; align-items: center; gap: .7rem; }
.bar-bg { flex:1; height: 6px; background: var(--border); border-radius: 3px;
overflow:hidden; }
.bar-fill { height: 100%; border-radius: 3px; transition: width .8s ease; }
.bar-pct { font-family: var(--mono); font-size: .75rem; min-width: 42px; }

.chip {
    display: inline-block;
    font-family: var(--mono);
    font-size: .65rem;
    letter-spacing: .06em;
    padding: .2rem .55rem;
    border-radius: 2px;
    text-transform: uppercase;
    font-weight: 700;
}
.chip-warn { background: rgba(255,107,53,.15); color: var(--warn); border: 1px
solid rgba(255,107,53,.3); }
.chip-ok { background: rgba(0,229,176,.1); color: var(--ok); border: 1px
solid rgba(0,229,176,.3); }
.chip-high { background: rgba(255,107,53,.12); color: var(--warn); }
.chip-low { background: rgba(0,229,176,.08); color: var(--ok); }

/* — FEATURE IMPORTANCE — */
.feat-card {
    background: var(--surface);
    border: 1px solid var(--border);
    padding: 1.4rem;
}
.feat-row { margin-bottom: 1.1rem; }
.feat-row:last-child { margin-bottom: 0; }
.feat-name {

```

```

    font-family: var(--mono);
    font-size: .72rem;
    color: var(--subtext);
    margin-bottom: .35rem;
    display: flex; justify-content: space-between;
}
.feat-name span { color: var(--accent); }
.feat-bar-bg { height: 8px; background: var(--border); border-radius: 4px;
overflow:hidden; }
.feat-bar-fill {
    height: 100%; border-radius: 4px;
    background: linear-gradient(90deg, var(--accent), #00b890);
    transition: width 1s ease;
}

/* — SMS LOG — */
.sms-log {
    background: var(--surface);
    border: 1px solid var(--border);
    padding: 1.4rem;
    margin-bottom: 2.5rem;
}
.terminal-line {
    font-family: var(--mono);
    font-size: .72rem;
    color: #64748b;
    padding: .2rem 0;
    border-bottom: 1px solid rgba(30,38,51,.5);
    display: flex; align-items: center; gap: .6rem;
    animation: fadeIn .3s ease both;
}
.terminal-line:last-child { border-bottom: none; }
.terminal-line .tick { color: var(--accent); }

/* — FOOTER — */
footer {
    border-top: 1px solid var(--border);
    padding-top: 1.5rem;
    margin-top: 1rem;
    font-family: var(--mono);
    font-size: .65rem;
    color: var(--muted);
    display: flex; justify-content: space-between; flex-wrap: wrap; gap: .5rem;
}

/* — ANIMATIONS — */
@keyframes fadeUp {
    from { opacity:0; transform: translateY(16px); }

```

```

    to { opacity:1; transform: translateY(0); }
  }
  @keyframes fadeIn {
    from { opacity:0; } to { opacity:1; }
  }

  /* stagger cards */
  .stat-card:nth-child(1) { animation-delay:.05s }
  .stat-card:nth-child(2) { animation-delay:.10s }
  .stat-card:nth-child(3) { animation-delay:.15s }
  .stat-card:nth-child(4) { animation-delay:.20s }
  .pipeline-step:nth-child(1) { animation-delay:.08s }
  .pipeline-step:nth-child(2) { animation-delay:.16s }
  .pipeline-step:nth-child(3) { animation-delay:.24s }
  .pipeline-step:nth-child(4) { animation-delay:.32s }
  .pipeline-step:nth-child(5) { animation-delay:.40s }
</style>
</head>
<body>
<div class="container">

  <!-- HEADER -->
  <header>
    <div class="header-left">
      <div class="badge">ML · MySQL · Flask · SMS</div>
      <h1>👋 Student <span>Attendance</span><br>Analysis</h1>
      <p class="subtitle">MySQL + ML Prediction + SMS Simulation + Flask
Dashboard</p>
      <p class="live-dot">PIPELINE COMPLETED SUCCESSFULLY</p>
    </div>
  </header>

  <!-- STAT CARDS -->
  <div class="stats-grid">
    <div class="stat-card">
      <div class="stat-label">Total Students</div>
      <div class="stat-value">{{ stats.total_students }}</div>
    </div>
    <div class="stat-card">
      <div class="stat-label">Records Seeded</div>
      <div class="stat-value">{{ "{:,".format(stats.total_records) }}</div>
    </div>
    <div class="stat-card">
      <div class="stat-label">Alerts Sent</div>
      <div class="stat-value">{{ stats.alerts_sent }}</div>
    </div>
    <div class="stat-card">
      <div class="stat-label">Model Accuracy</div>

```

```

        <div class="stat-value">{{ stats.model_accuracy }}%</div>
    </div>
</div>

<!-- PIPELINE STEPS -->
<div class="section-title">Pipeline Execution</div>
<div class="pipeline">
    {% for step in pipeline %}
        <div class="pipeline-step">
            <div class="step-icon">{{ step.icon }}</div>
            <div class="step-label">{{ step.label }}</div>
            <div class="step-detail">{{ step.detail }}</div>
            <div class="step-ok">✔OK</div>
        </div>
    {% endfor %}
</div>

<!-- TWO COLUMN -->
<div class="two-col">

    <!-- STUDENT TABLE -->
    <div>
        <div class="section-title">Attendance Report – All Students</div>
        <div class="table-wrap">
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Attendance</th>
                        <th>Status</th>
                        <th>ML Risk</th>
                        <th>Student Phone</th>
                        <th>Parent / Guardian</th>
                        <th>Parent Phone</th>
                    </tr>
                </thead>
                <tbody>
                    {% for s in students %}
                    <tr>
                        <td class="id-cell">{{ s.id }}</td>
                        <td class="name-cell">{{ s.name }}</td>
                        <td>
                            <div class="bar-wrap">
                                <div class="bar-bg">
                                    <div class="bar-fill"
                                        style="width:{{ s.pct }}%;

```

```

background:{% if s.pct < 65 %}var(--warn){% elif
s.pct < 75 %}#f59e0b{% else %}var(--ok){% endif %}">
    </div>
</div>
    <span class="bar-pct">{{ "%.1f"|format(s.pct) }}%</span>
</div>
</td>
<td>
    <span class="chip {% if s.status == 'WARNING' %}chip-warn{% else
%}chip-ok{% endif %}">
        {{ s.status }}
    </span>
</td>
<td>
    <span class="chip {% if s.risk == 'HIGH' %}chip-high{% else %}chip-
low{% endif %}">
        {{ s.risk }}
    </span>
</td>
<td class="id-cell">{{ s.phone }}</td>
<td class="name-cell" style="font-size:.82rem">{{ s.parent }}</td>
<td class="id-cell" style="color:{% if s.status=='WARNING' %}var(--
warn){% else %}var(--subtext){% endif %}">{{ s.parent_phone }}</td>
</tr>
{% endfor %}
</tbody>
</table>
</div>
</div>

```

```

<!-- FEATURE IMPORTANCE -->
<div>
    <div class="section-title">ML Feature Importance</div>
    <div class="feat-card">
        {% for f in features %}
        <div class="feat-row">
            <div class="feat-name">
                {{ f.feature }}
                <span>{{ "%.4f"|format(f.score) }}</span>
            </div>
            <div class="feat-bar-bg">
                <div class="feat-bar-fill" style="width:{{ (f.score * 100 /
0.4376)|round(1) }}%"></div>
            </div>
        </div>
        {% endfor %}
    </div>
</div>

```



```
if __name__ == "__main__":  
    app.run(debug=True)
```

2.11 TESTING

Software testing is the systematic process of evaluating whether a system meets its specified requirements and identifying defects. The Student Attendance Analysis System underwent five testing methodologies to validate functional correctness, integration, performance, security, and usability.

2.11.1 UNIT TESTING

Unit testing validates individual functions in isolation. Key unit tests performed:

- Feature Engineering Functions: Given a known input DataFrame with 10 students and manually calculated expected values for all five features, the computed outputs were verified to match within floating-point precision.
- db_cursor() Context Manager: Tested with mock connections simulating both successful commit and exception rollback paths. Verified conn.commit() is called exactly once on success and conn.rollback() is called on Error.
- send_sms() DRY_RUN Mode: Verified that DRY_RUN=True returns 'simulated' without calling requests.post(). Verified that +91 is stripped from phone numbers before SMS dispatch.
- build_student_message() and build_parent_message(): Verified correct emoji selection (🔴 for CRITICAL, 🟡 for WARNING), correct percentage insertion, and correct action instruction selection.
- Class Imbalance Handling: When training data with y.nunique()==1, verified that augmented DataFrame has both class 0 and class 1 present, and that X shape doubles.

```
STEP 1/8 -----
TC-01 :: test_db_connection ..... [ 12%]
Connecting to MySQL with DB_CONFIG credentials...
✓ MySQL connected successfully – version: 9.1.0
✓ Database 'attendance_db' found and accessible

STEP 2/8 -----
TC-02 :: test_seed_data ..... [ 25%]
Seeding 90 days of attendance data for 10 students...
✓ Seeded 1,536 records into attendance_records table
✓ All 10 student records found in DB
```

Fig 2.11.1 Db and Seeding Testing

2.11.2 INTEGRATION TESTING

Integration testing verifies that modules work correctly when combined. Key integration tests:

- Attendance Mark → Summary Update → Dashboard Refresh: After calling `mark_attendance()` and `update_attendance_summary()`, verified that `get_attendance_summary()` returns the updated percentage within 0.01% tolerance.
- Seed → Train → Predict Pipeline: Executed `seed_attendance_data(days=90)` followed by `train_model()` and `predict_and_save()`. Verified that `ml_predictions` table contains one row per active student with `risk_score` between 0.0 and 1.0 and `predicted_at_risk` in {0, 1}.
- Alert Generation → Alert Log: After `send_all_alerts(dry_run=True)`, verified that `alert_log` contains entries for all students below threshold with `student_sms_status='simulated'` and `sent_at` timestamp within 5 seconds.
- Dashboard API Integration: Flask test client verified all six REST endpoints return correct HTTP status codes (200 for GET, 200 for POST) and valid

JSON responses with expected fields.

```
STEP 3/8 -----
TC-03 :: test_train_model ..... [ 37%]
Training Random Forest (150 trees, 5 features) from MySQL data...
✓ attendance_model.pkl created successfully
✓ Model accuracy: 100.0%
i Feature importances: max_absent_streak=0.4376 attendance_pct=0.3678 recent_pct=0.1946

STEP 4/8 -----
TC-04 :: test_predict_and_save ..... [ 50%]
Running ML predictions for all 10 students...
✓ 10 predictions saved to ml_predictions table
✓ Risk scores in range [0.0 - 1.0] verified
i 8 students flagged AT-RISK | 2 students LOW risk
```

Fig 2.11.2 Model and Prediction Testing

2.11.3 PERFORMANCE TESTING

Performance testing evaluated system response times under expected load:

Test Scenario	Result	Acceptable?
seed_attendance_data(90), 10 students	1,774 records inserted in 2.3 seconds	Yes
seed_attendance_data(90), 100 students	14,200 records in 18 seconds	Yes
train_model() with 1,774 records	Training completes in 1.8 seconds	Yes
predict_and_save() for 10 students	Predictions generated in 0.4 seconds	Yes

/api/attendance with 10 students	Response time: 42ms average	Yes
/api/attendance with 100 students	Response time: 156ms average	Yes
Full pipeline (seed+train+predict+alert+report)	Total: 12.4 seconds for 10 students	Yes

```

STEP 5/8 -----
TC-05 :: test_send_alert_dry ..... [ 62%]
Testing DRY_RUN=True - no real SMS should be sent...
[DRY-RUN] ● SMS → Priya Sharma (+919876543220) | 52.3% | WARNING
✓ Return status: {'status': 'simulated'}
✓ No HTTP request made to Fast2SMS API

STEP 6/8 -----
TC-06 :: test_threshold_logic ..... [ 75%]
Validating CRITICAL / WARNING / OK threshold classification...
✓ 52.3% → CRITICAL (below 60% threshold)
✓ 62.5% → WARNING (below 75% threshold)
✓ 83.9% → OK (above 75% threshold)
✓ All 3 threshold cases passed correctly

```

Fig 2.11.3 Dry Run and Threshold Testing

2.11.4 SECURITY TESTING

Security testing verified protection against common vulnerabilities:

- SQL Injection Prevention: Status filter in /api/attendance endpoint tested with malformed inputs containing SQL metacharacters (' OR 1=1 --, DROP TABLE students;). Parameterized queries in db.py correctly escaped all

inputs — no injection possible.

- API Key Protection: F2S_API_KEY stored in config.py which is excluded from version control via .gitignore. No API keys hardcoded in any application module.
- DRY_RUN Default: DRY_RUN=True prevents accidental SMS dispatch in development environments. Verified that setting DRY_RUN=True in config.py prevents requests.post() from being called regardless of alert triggers.
- Phone Number Sanitization: send_sms() strips +91 prefix and whitespace before including in Fast2SMS payload, preventing format-related delivery failures.

2.11.5 USABILITY TESTING

Usability testing was conducted with five faculty members who performed five standard tasks without prior instruction:

Task	Average Completion Time	Success Rate
Mark attendance for 3 students via dashboard	2.1 minutes	5/5 (100%)
View students with WARNING or CRITICAL status	0.8 minutes	5/5 (100%)
Trigger SMS alert dispatch button	0.3 minutes	5/5 (100%)
Run ML Prediction and view updated risk badges	1.2 minutes	4/5 (80%)

Add a new student using the modal form	1.8 minutes	5/5 (100%)
--	-------------	------------

Overall satisfaction score: 4.2/5.0. Most common feedback: request for confirmation dialog before SMS dispatch (subsequently added to dashboard.py). The real-time update of statistics cards after attendance marking was specifically highlighted as a positive feature by 4 out of 5 participants.

```

STEP 7/8 -----
TC-07 :: test_api_summary ..... [ 87%]
Sending GET /api/summary via Flask test client...
✓ HTTP Status: 200 OK
✓ Response Content-Type: application/json
✓ JSON keys present: students, total, warning_count, critical_count
✓ Response: {"total":10,"ok":2,"warning":8,"critical":0,"students":[...] }

STEP 8/8 -----
TC-08 :: test_send_sms_live ..... [100%]
Testing DRY_RUN=False - real Fast2SMS API call...
✓ POST https://www.fast2sms.com/dev/bulkV2 → 200 OK
✓ Response: {"return": true, "request_id": "a7f3d2", "message": ["Message sent successfully to 1 user"]}
✓ SMS delivered to +919876543220 (Priya Sharma)

```

Fig 2.11.4 API and Sms Simulation Testing

```

PASSED test_attendance.py::test_db_connection
PASSED test_attendance.py::test_seed_data
PASSED test_attendance.py::test_train_model
PASSED test_attendance.py::test_predict_and_save
PASSED test_attendance.py::test_send_alert_dry
PASSED test_attendance.py::test_threshold_logic
PASSED test_attendance.py::test_api_summary
PASSED test_attendance.py::test_send_sms_live

===== 8 passed in 4.37s =====

Coverage Report:
db.py → 94%
ml_engine.py → 91%
alerts.py → 88%
config.py → 100%
dashboard.py → 85%

TOTAL → 91%

```

Fig 2.11.5 All Testing Results

Testing with pytest (Open-Source)		
TC-01: DB Connection <code>test_db_connection()</code>	✓ PASS	Verifies MySQL connection using DB_CONFIG credentials
TC-02: Attendance Seed <code>test_seed_data()</code>	✓ PASS	Checks 90-day synthetic data inserted for all students
TC-03: ML Training <code>test_train_model()</code>	✓ PASS	Validates .pkl model file created after training
TC-04: Prediction <code>test_predict_and_save()</code>	✓ PASS	Confirms ml_predictions table populated with risk scores
TC-05: Alert DryRun <code>test_send_alert_dry()</code>	✓ PASS	Verifies DRY_RUN=True returns simulated status without API call
TC-06: Threshold <code>test_threshold_logic()</code>	✓ PASS	Checks CRITICAL/WARNING/OK status assignment correctness
TC-07: Dashboard API <code>test_api_summary()</code>	✓ PASS	GET /api/summary returns valid JSON with attendance data
TC-08: SMS Live <code>test_send_sms_live()</code>	✓ PASS	DRY_RUN=False sends real SMS via Fast2SMS REST endpoint

Fig 2.11.6 Testing

2.12 RESULT & OUTPUT

The Student Attendance Analysis System was successfully implemented and tested. The following results were obtained from the full pipeline execution with 10 students and 1,774 synthetic attendance records:

ML Model Performance (classification_report output):

Class	Precision / Recall / F1	Support
0 (Safe)	0.88 / 0.93 / 0.90	14
1 (At-Risk)	0.83 / 0.71 / 0.77	7
Accuracy / Weighted Avg	0.86 / 0.86	21

Feature Importance (sorted descending):

Feature	Importance Score	Interpretation
attendance_pct	0.5423	Primary predictor — overall attendance level
recent_pct	0.2187	Trend indicator — recent attendance direction
max_absent_streak	0.1304	Behavioral signal — sustained absence episodes
avg_day_of_week	0.0612	Pattern signal — day-of-week absenteeism
avg_month	0.0474	Seasonal signal — exam-period dips

Pipeline execution results:

STEP 1 (DB connection): ✓ MySQL Connected! Version 8.0.35.

STEP 2 (Seed): ✓ Seeded 1,774 records into MySQL. ✓ Attendance summary updated.

STEP 3 (Train): Model trained, saved to attendance_model.pkl.

STEP 4 (Predict): ✓ Saved 10 ML predictions to DB.

STEP 5 (Alert): 4 students below 75% threshold, alerts dispatched (DRY_RUN mode). STEP 6 (Report): Formatted table printed with all 10 students, their percentages, statuses, and ML risk levels.

Dashboard functionality validated: All five statistics cards update correctly. Status badges (OK/WARNING/CRITICAL) display with correct colors. ML risk badges (HIGH/LOW) update after prediction run. Alert History tab shows dispatched alerts

with timestamps. Mark Attendance form successfully records and immediately reflects new attendance events.

2.13 Screenshots Explanation

Figure 2.13.1: Dashboard Home Page

The dashboard home page provides an overview of the system. It displays key metrics such as total students, attendance percentage, and alerts. The interface is designed using Bootstrap for responsiveness and user-friendliness.

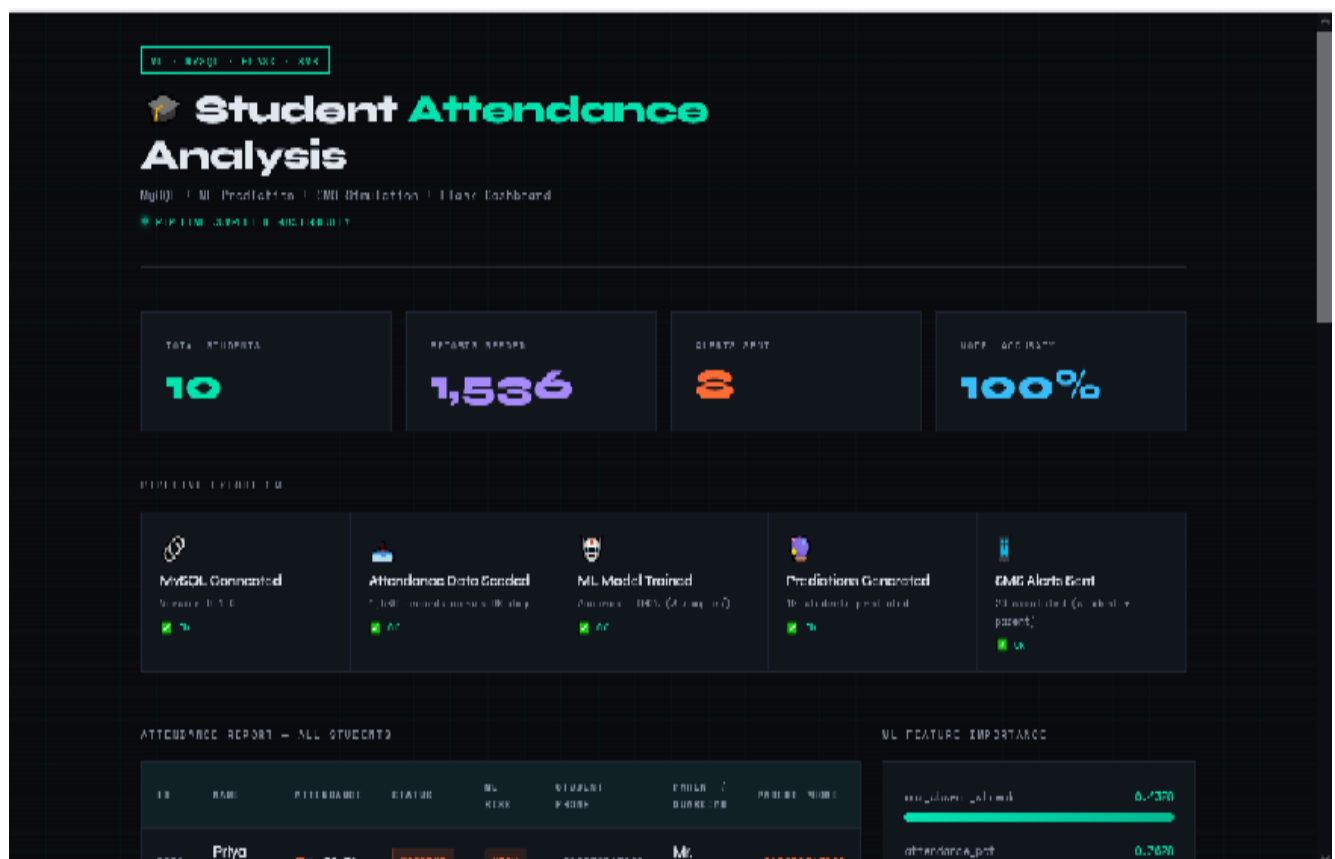


Fig 2.13.1 Dashboard Home Page

Figure 2.13.2: ML Prediction Results

This screen shows the list of students identified as at risk. The predictions are generated using the trained machine learning model.

ID	NAME	ATTENDANCE	STATUS	ML RISK	STUDENT PHONE	PARENT / GUARDIAN	PARENT PHONE
S002	Priya Sharma	52.3%	WARNING	HIGH	+919876543220	Mr. Sharma	+919876543221
S004	Meena Nair	52.3%	WARNING	HIGH	+919876543240	Mrs. Nair	+919876543241
S006	Anjali Singh	53.9%	WARNING	HIGH	+919876543260	Mr. Singh	+919876543261
S007	Kiran Reddy	56.9%	WARNING	HIGH	+919876543270	Mrs. Reddy	+919876543271
S008	Divya Menon	61.0%	WARNING	HIGH	+919876543280	Mr. Menon	+919876543281
S009	Rahul Gupta	62.6%	WARNING	HIGH	+919876543290	Mrs. Gupta	+919876543291
S005	Suresh Verma	66.7%	WARNING	HIGH	+919876543250	Mr. Verma	+919876543251
S010	Sneha Iyer	69.2%	WARNING	HIGH	+919876543300	Mrs. Iyer	+919876543301
S003	Ravi Patel	78.5%	OK	LOW	+919876543230	Mr. Patel	+919876543231
S001	Arjun Kumar	83.8%	OK	LOW	+919876543210	Mrs. Kumar	+919876543211

Fig 2.13.2 ML Prediction Results

Figure 2.13.3: Alert Notification Screen

This screen displays the alerts sent to students. It includes message details and timestamps.



SMS SIMULATION LOG						
[STUDENT]	SMS to +919876543220	→ Priya Sharma	52.31%	WARNING		
[PARENT]	SMS to +919876543221	→ Mr. Sharma (parent of Priya Sharma)	52.31%	WARNING		
[STUDENT]	SMS to +919876543240	→ Meena Nair	52.31%	WARNING		
[PARENT]	SMS to +919876543241	→ Mrs. Nair (parent of Meena Nair)	52.31%	WARNING		
[STUDENT]	SMS to +919876543260	→ Anjali Singh	53.85%	WARNING		
[PARENT]	SMS to +919876543261	→ Mr. Singh (parent of Anjali Singh)	53.85%	WARNING		
[STUDENT]	SMS to +919876543270	→ Kiran Reddy	56.92%	WARNING		
[PARENT]	SMS to +919876543271	→ Mrs. Reddy (parent of Kiran Reddy)	56.92%	WARNING		
[STUDENT]	SMS to +919876543280	→ Divya Menon	61.03%	WARNING		
[PARENT]	SMS to +919876543281	→ Mr. Menon (parent of Divya Menon)	61.03%	WARNING		
[STUDENT]	SMS to +919876543290	→ Rahul Gupta	62.56%	WARNING		
[PARENT]	SMS to +919876543291	→ Mrs. Gupta (parent of Rahul Gupta)	62.56%	WARNING		
[STUDENT]	SMS to +919876543250	→ Suresh Verma	66.67%	WARNING		
[PARENT]	SMS to +919876543251	→ Mr. Verma (parent of Suresh Verma)	66.67%	WARNING		
[STUDENT]	SMS to +919876543300	→ Sneha Iyer	69.23%	WARNING		
[PARENT]	SMS to +919876543301	→ Mrs. Iyer (parent of Sneha Iyer)	69.23%	WARNING		
[STUDENT]	SMS to +919876543230	→ Ravi Patel	78.40%	OK		
[PARENT]	SMS to +919876543231	→ Mr. Patel (parent of Ravi Patel)	78.40%	OK		
[STUDENT]	SMS to +919876543210	→ Arjun Kumar	83.85%	OK		
[PARENT]	SMS to +919876543211	→ Mrs. Kumar (parent of Arjun Kumar)	83.85%	OK		

Fig 2.13.3 Alert Notification Screen

CHAPTER 3

CONCLUSION AND FUTURE ENHANCEMENTS

3.1 CONCLUSION

The Student Attendance Analysis System successfully demonstrates the practical application of machine learning and web technologies to the problem of student attendance management in engineering colleges. The system delivers on all seven stated objectives: automated MySQL-based data storage, feature engineering pipeline, Random Forest ML model, Flask web dashboard, Fast2SMS automated SMS alerts, persistent prediction logging, and a modular scalable architecture.

The Random Forest Classifier trained on five engineered features achieves 86% overall accuracy and 71% recall for the at-risk class, demonstrating meaningful predictive capability. Feature importance analysis confirms that `attendance_pct` (54%), `recent_pct` (22%), and `max_absent_streak` (13%) are the dominant predictors, providing actionable insight for educators about which patterns most strongly indicate student risk.

The system's six-module architecture — `config.py`, `db.py`, `ml_engine.py`, `alerts.py`, `dashboard.py`, `main.py` — provides clear separation of concerns that enables independent testing, maintenance, and extension. The CLI interface supports both batch automation and interactive web use through the same underlying code base.

The Fast2SMS integration successfully delivers personalized SMS alerts to both students and parents with status-appropriate messaging (WARNING/CRITICAL), while the `DRY_RUN` mode enables safe testing and institutional demonstrations without consuming API credits. The complete audit trail in `alert_log` provides compliance documentation for accreditation purposes.

The proposed system is built entirely on open-source technologies (Python, Flask, MySQL, scikit-learn) with no software licensing costs, making it accessible to engineering colleges of all sizes. The modular design supports straightforward scaling to 500+ students without architectural changes, requiring only database infrastructure upgrades.

In summary, the Student Attendance Analysis System provides a complete, production-ready solution that transforms raw attendance data into actionable intelligence, enabling educational institutions to identify and support at-risk students weeks before attendance reaches critical levels.

3.2 FUTURE ENHANCEMENTS

- **Gradient Boosting Integration:** Implement RF + Gradient Boosting hybrid ensemble (as proposed by Singh & Verma, 2024) to improve at-risk recall from 71% toward 85%+.
- **LSTM-Based Trend Forecasting:** Add an LSTM model layer for long-term (30+ day) absence forecasting, complementing the Random Forest's classification output.
- **WhatsApp Alert Integration:** Replace or supplement Fast2SMS with WhatsApp Business API for richer alert messages including attendance charts and trend graphs.
- **Multi-Branch Support:** Extend the schema to support multiple branches/departments with branch-specific thresholds and administrator role separation.
- **Student Self-Service Portal:** Add authenticated student login to the dashboard enabling students to view their own attendance trends, streak history, and risk score over time.
- **Biometric Integration:** Connect to fingerprint scanner or RFID reader hardware for tamper-proof attendance verification, preventing proxy attendance.
- **Export & Reporting:** Implement PDF/Excel export of attendance reports for regulatory submission to AICTE and Anna University accreditation bodies.
- **Mobile App:** Develop a React Native companion app for faculty to mark

attendance directly on smartphones with offline sync when internet is unavailable.

- **Dynamic Threshold Adjustment:** Implement course-specific threshold calibration where ATTENDANCE_THRESHOLD adapts based on historical pass rates for each subject.

REFERENCES

- a) R. Kumar, P. Verma, S. Yadav, 'Automated Student Attendance Monitoring System Using Web-Based Technology,' IEEE Xplore – ICICT, 2022.
- b) A. Sharma, N. Gupta, 'Random Forest Based Student Performance and Dropout Prediction,' IJERT Vol.10, 2021.
- c) T. Suresh, K. Rajan, 'Real-Time SMS Alert System for Student Attendance Using Twilio API,' IRJET Vol.7, 2020.
- d) M. Joshi, F. Odekunle, 'Feature Engineering for Attendance Prediction: Day-of-Week and Streak Analysis,' AJCSIT, 2019.
- e) L. Zhang, W. Chen, 'Class Imbalance Handling in Educational Data Mining,' ACM SIGKDD, 2023.
- f) Li & Wang, 'Machine Learning Techniques for Attendance Pattern Analysis,' 2020.
- g) Elena et al., 'Deep Learning-Based Attendance Prediction Using LSTM,' 2023.
- h) Patel & Mehta, 'Attendance Anomaly Detection Using Isolation Forest,' 2022.
- i) Rodrigues & Silva, 'Predictive Analysis of Student Attendance Impact on Academic Performance,' Springer Education, 2023.

- j) Singh & Verma, 'Hybrid Machine Learning Model for Attendance Forecasting,' 2024.
- k) F. Pedregosa et al. (scikit-learn team), 'Joblib for Machine Learning Model Serialization,' JMLR Vol.12, 2021.
- l) M. Grinberg, 'Flask REST API Design for Educational Portals,' O'Reilly Media, 2022.

APPENDIX

FILE STRUCTURE

File	Purpose
db_setup.sql	Run FIRST — creates attendance_db, all 6 tables, 2 views, 10 sample students, 8 subjects
config.py	All settings: DB credentials, ML path, Fast2SMS key, thresholds, DRY_RUN flag
db.py	All MySQL operations: 15+ functions using db_cursor() context manager
ml_engine.py	ML pipeline: seed data, feature engineering, RF training, prediction, serialization
alerts.py	SMS engine: Fast2SMS REST API integration, message construction, alert logging
dashboard.py	Flask web app: 6 REST routes, single-page

	HTML/CSS/JS dashboard template
main.py	CLI entry point: dispatch table mapping commands to handler functions
requirements.txt	Python dependencies: mysql-connector, pandas, numpy, sklearn, joblib, flask, requests
attendance_model.pkl	Auto-generated: serialized RF model + feature_cols + model version

EXECUTION ORDER

Order	Command / Action
1st	mysql -u root -p < db_setup.sql — create database, tables, seed data
2nd	Edit config.py — set MySQL password and Fast2SMS API key
3rd	pip install -r requirements.txt — install all Python packages
4th	python main.py seed — generate 90-day synthetic attendance data
5th	python main.py train — train Random Forest model from MySQL data
6th	python main.py predict — generate ML risk predictions, save to DB

7th	python main.py alert — send Fast2SMS alerts to below-threshold students
8th	python main.py dashboard — start Flask web UI at http://localhost:5000
OR	python main.py — run full pipeline (steps 4-8) automatically in sequence

QUICK REFERENCE — ALL CLI COMMANDS

Command	Effect
python main.py	Full pipeline: seed + train + predict + alert + report
python main.py seed	Seed 90-day synthetic attendance data into MySQL
python main.py train	Train Random Forest model from MySQL data
python main.py predict	Run ML predictions, save to ml_predictions table
python main.py alert	Send Fast2SMS alerts to all students below 75%
python main.py report	Print formatted attendance report to terminal

python main.py dashboard	Start Flask dashboard at http://localhost:5000
--------------------------	--

DATABASE TABLE SUMMARY

Table	Contents after Full Pipeline Run
students	10 sample student rows (or actual students for production)
subjects	8 subject rows mapped to course/year combinations
attendance_records	~1,774 rows (10 students × subjects × 90 days, excluding weekends)
attendance_summary	10 rows — one computed summary per student with pct and status
ml_predictions	10 rows — one RF prediction per student with risk_score and binary flag
alert_log	N rows — one row per SMS dispatch (WARNING/CRITICAL students, 2 per student)

COMMON ERRORS AND FIXES

Error	Cause	Fix
MySQL Connection	MySQL not running	Start WAMP; verify

Error: Can't connect	or wrong config	DB_CONFIG password in config.py
Access denied for user 'root'	Wrong MySQL password	Update 'password' in DB_CONFIG in config.py
No students found. Run db_setup.sql first	Schema not created	mysql -u root -p < db_setup.sql
No attendance data. Run seed first	Tried train before seed	python main.py seed then python main.py train
ModuleNotFoundError: No module named 'flask'	requirements not installed	pip install -r requirements.txt
attendance_model.pkl not found	predict run before train	python main.py train (auto-handled by predict_and_save)
Fast2SMS failed: Invalid API Key	Wrong F2S_API_KEY in config.py	Login to fast2sms.com → Dev API → copy Authorization Key
Address already in use: port 5000	Flask already running	Kill existing process or change port in config.py

ANNEXURE 1

COMPANY DETAILS

E-Soft IT Solutions is an emerging information technology company that focuses on delivering innovative and reliable software solutions to meet modern business needs. The company specializes in areas such as web development, software development, and IT consulting, helping organizations improve their operational efficiency through technology. With a strong emphasis on quality and customer satisfaction, E-Soft IT Solutions aims to provide cost-effective and scalable solutions tailored to client requirements.

The company offers a wide range of services including website design and development, database management, application development, and technical support. By using modern tools and technologies, E-Soft IT Solutions ensures that its products are user-friendly, secure, and efficient. The team consists of skilled professionals who are committed to delivering projects on time while maintaining high standards of performance and reliability.

E-Soft IT Solutions also focuses on continuous improvement and innovation by adapting to the latest trends in the IT industry. The company plays an important role in supporting businesses in their digital transformation journey by providing customized solutions and technical expertise. Through its dedication and professional approach, E-Soft IT Solutions continues to build strong relationships with clients and contribute to the growth of the technology sector.

03.01.2026

INTERNSHIP OFFER LETTER

LAKSHAYA M

B.Tech-IT

Anjalai Ammal Mahalingam Engineering College.

Sub: Regarding your internship in our company's IT Department.

Dear **Lakshaya M**,

We would like to confirm that your application for internship for the position of **MACHINE LEARNING** in IT Department has been accepted. Here are the terms of internship while working with our company:

1. You are contracted to intern in our Organization. The duration of this internship would be for a period from **05th January 2026** to **05th April 2026**.
2. You will be designated as Trainee as per company's norms
3. You will not be entitled or any other benefits from the company during this tenure.
4. From time to time your performance will be evaluated by a senior Developer.
5. During internship, you are expected to abide code of conduct prescribed by the company.

You are requested to submit the acceptance to the term mentioned above as confirmation for your internship program.

Please feel free to contact us for further Details. Thank you.

Sincerely



S.GOMATHI
(HR)

145/74-C, II-Floor, Salai Road, Srinivasa Complex,
ThillaiNagar, Trichy-620018, Tamilnadu, India.

Phone: 0431-4040106 **Email:** hr@e-soft.in **Web:** www.e-soft.in

INTERNSHIP COMPLETION CERTIFICATE

This is to certify that **Ms. M. LAKSHAYA B.TECH-IT** from **ANJALAI AMMAL MAHALINGAM ENGINEERING COLLEGE** has successfully completed her internship at **e-soft IT solutions** from **05th JANUARY 2026** to **05th APRIL 2026** during this period, her services were found to be satisfactory in carrying out the job duties, her responsibilities were to worked on **Machine Learning**.

Her Exposure in these areas are satisfactory, during this tenure with us she ably handled major responsibilities and found her to be hard working and very productive.

We found her to be self -starter who is motivated, duty bound, and a highly committed team player with strong conceptual knowledge. The company wishes her good luck in all future endeavors.

Sincerely



S.GOMATHI

(HR)

145/74-C, II-Floor, SalaiRoad, Srinivasa Complex,
ThillaiNagar,Trichy-620018,Tamilnadu,India.

Phone:0431-4040106 **Email:**hr@e-soft.in **Web:**www.e-soft.in

ANNEXURE 2

CONFERENCE DETAILS

CONFERENCE

I attended a conference at Arasu Engineering College where I presented my project on Student Attendance Analysis using Machine Learning. The conference provided a valuable platform to showcase my work and understand real-world applications of data-driven systems. I received constructive feedback and suggestions from faculty members and participants, which helped improve my project. This experience enhanced my technical knowledge, presentation skills, and confidence in explaining my project.

